Java Database Connectivity (JDBC)

#	Scenario Title / Use Case	Level	Target Role
1	Fetch books in a price band that are currently available	L2	Java Backend Dev
2	Find room bookings that overlap a requested time window	L3	Backend Dev / DB Engineer
3	Show recent bookings for a customer in the past 7 days	L1	Junior Java Dev
4	Fetch reviews for a dish sorted by rating	L1	Junior Java Dev
5	Fetch transactions of a given type for a user	L1	Junior Java Dev
6	Prevent duplicate enrollments in a course	L2	Backend Dev
7	Calculate average restaurant rating	L1	Junior Java Dev
8	Prevent duplicate seat bookings	L2	Backend Dev
9	Fetch failed payments of a customer	L1	Support Tools Dev / Java Dev
10	Insert new library member securely	L1	Junior Java Dev
11	Atomic wallet fund transfer with transactions	L4	Backend/FinTech Engineer
12	Display all travelers in tabular format	L1	Junior Java Dev
13	Update inventory stock after book purchase	L2	Backend Dev
14	Update tenant’s contact details	L1	Junior Java Dev
15	Check event ticket availability	L1	Junior Java Dev
16	Insert a product review securely	L1	Junior Java Dev
17	Insert a candidate’s job application (return ID)	L2	Backend Dev
18	Delete a user and their purchases atomically	L3	Backend Dev
19	Retrieve a user’s reading history	L1	Junior Java Dev
20	Retrieve a user’s flight bookings	L1	Junior Java Dev
21	Fetch reservations for a user within a date range	L2	Backend Dev
22	Calculate total expenses in the last 30 days	L1	Junior Java Dev
23	Update ticket count after booking	L2	Backend Dev

24	Retrieve low-stock products for restocking	L1	Ops/Reporting Dev
25	Check if a user’s subscription is active	L1	Junior Java Dev
26	Retrieve available books in a library	L1	Junior Java Dev
27	Update order status to "Pending"	L1	Junior Java Dev
28	Update loyalty points after a purchase	L2	Backend Dev
29	Insert new product feedback	L1	Junior Java Dev
30	Fetch overdue tasks for a user	L1	Junior Java Dev
31	Migrate staff data (batched + transactional)	L4	Data Eng / Backend Dev
32	Fetch a user’s invoices from previous calendar month	L2	Backend Dev
33	Update a user’s profile information	L1	Junior Java Dev
34	Fetch all unread notifications for a user	L1	Junior Java Dev
35	Calculate average product rating	L1	Junior Java Dev
36	Retrieve active subscribers for a plan	L1	Junior Java Dev
37	Retrieve all out-of-stock products	L1	Junior Java Dev / Ops
38	Retrieve tasks for a user ordered by priority	L1	Junior Java Dev
39	Retrieve movies of a genre released in past year	L1	Junior Java Dev
40	Update loyalty points for a customer after purchase	L2	Backend Dev
41	Insert an employee and log the action atomically	L3	Backend Dev
42	Call a stored procedure using CallableStatement	L2	Backend Dev
43	Batch update multiple students’ GPA efficiently	L3	Backend Dev / Data Eng
44	Use a JDBC connection pool (DataSource)	L3	Backend Dev / SRE
45	Scrollable & updatable ResultSet for library records	L3	Backend Dev / Desktop
46	Prevent SQL injection in flight search	L1	Junior Java Dev
47	Savepoint for partial rollback in payments	L4	Backend/FinTech Engineer
48	Stored procedure with OUT parameter (applicant count)	L2	Backend Dev
49	Batch-insert shipment tracking updates from CSV	L3	Data Eng / Backend Dev

50	All-or-nothing fund transfers (transactional batch)	L4	Backend/FinTech Engineer
51	Call stored procedure to get employee details	L2	Backend Dev
52	Scrollable ResultSet for browsing books	L2	Backend Dev / Desktop
53	Preventing SQL injection in legacy query building	L1	Junior Java Dev
54	Bulk price updates from CSV (batched + txn)	L3	Data Eng / Backend Dev
55	Scrollable + updatable ResultSet for inline editing	L3	Backend Dev / Desktop
56	Generate schema docs via DatabaseMetaData/RSMeta	L3	Tools/Platform Eng
57	Stored procedure with IN & OUT (discount calc)	L2	Backend Dev
58	Partial rollback with JDBC savepoints	L4	Backend Dev
59	Efficient connection management with a JDBC pool	L3	Backend Dev / SRE
60	Store & retrieve images (BLOB) with JDBC	L3	Backend Dev
61	Stream millions of rows without blowing memory	L4	Data Eng / Backend Dev
62	Close JDBC resources reliably (no leaks)	L2	Backend Dev
63	Fast bulk inserts with PreparedStatement batching	L3	Data Eng / Backend Dev
64	Prevent SQL Injection in JDBC (login/signup)	L1	Junior Java Dev
65	Work offline using RowSet (CachedRowSet)	L3	Desktop/Tools Dev
66	Use Savepoints for Partial Rollback in JDBC	L4	Backend Dev
67	Efficient Batch Inserts for Shipments	L3	Data Eng / Backend Dev
68	Retrieve DB metadata for auditing	L2	Platform/Infra Dev
69	Dynamically read any query’s columns/values	L2	Tools/Analytics Dev
70	Call stored proc with IN/OUT (getStudentGPA)	L2	Backend Dev
71	Secure product search with LIKE (inj-safe)	L1	Junior Java Dev

Scenario 1: Fetch books in a price band that are currently available

In a library app (think e-commerce browsing), readers filter by price and only want items that are actually borrowable. We need a parameterized query that returns books with price between min/max and stock_count > 0.

Answer:

I'd use a PreparedStatement with BETWEEN for the price band and a strict stock_count > 0 predicate. I'll also make the method accept min/max so it can be reused. Even if the column is DOUBLE, I prefer handling money as BigDecimal in Java to avoid floating-point surprises. I'll also sort by price and title for predictable UI ordering.

Code:

```
import java.math.BigDecimal;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

class Book {
    final int bookId;
    final String title;
    final BigDecimal price;
    final int stock;

    Book(int bookId, String title, BigDecimal price, int stock) {
        this.bookId = bookId;
        this.title = title;
        this.price = price;
        this.stock = stock;
    }

    @Override public String toString() {
        return "Book{id=%d, title='%s', price=%s, stock=%d}".formatted(bookId, title,
price, stock);
    }
}

public class FetchBooksByPriceRange {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/library";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        List<Book> result = findBooksInPriceRange(new BigDecimal("100.00"), new
BigDecimal("500.00"));
        result.forEach(System.out::println);
    }
```

```
    public static List<Book> findBooksInPriceRange(BigDecimal minPrice,
BigDecimal maxPrice) {
        String sql = """
            SELECT book_id, title, price, stock_count
            FROM books
            WHERE price BETWEEN ? AND ?
              AND stock_count > 0
            ORDER BY price ASC, title ASC
        """;

        List<Book> books = new ArrayList<>();

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            // If your column is DECIMAL, prefer setBigDecimal. If it's DOUBLE,
setDouble is fine.
            ps.setBigDecimal(1, minPrice);
            ps.setBigDecimal(2, maxPrice);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    int id = rs.getInt("book_id");
                    String title = rs.getString("title");
                    BigDecimal price = rs.getBigDecimal("price"); // reads DECIMAL
precisely
                    int stock = rs.getInt("stock_count");
                    books.add(new Book(id, title, price, stock));
                }
            }
        } catch (SQLException e) {
            // In production, log with context, don't just print stack trace
            e.printStackTrace();
        }
        return books;
    }
}
```

Reasoning to interviewer:

- PreparedStatement avoids SQL injection and lets the DB cache the plan.
- stock_count > 0 enforces availability at the database layer, so we don't fetch dead data and filter in Java.
- BigDecimal is safer for money; if the DB type is DECIMAL, you get exact arithmetic end-to-end.

• ORDER BY price, title provides stable results for pagination/UI caching.

• The method returns domain objects instead of printing in the query loop—easier to test and compose.

Future scope:

• Add pagination (LIMIT/OFFSET) and a covering index on (price, stock_count, book_id).

• Expose author filters (author_id IN …).

• If the catalog is large, consider a read replica for browse traffic.

Scenario 2: Find room bookings that overlap a requested time window

In a co-working scheduler (like airline seat allocation), we must block double-booking if any part of an existing booking intersects the requested slot. The clean overlap rule is: an existing interval [start, end) overlaps the requested [reqStart, reqEnd) when start < reqEnd AND end > reqStart.

Answer:

I'll accept roomId, reqStart, reqEnd as parameters and query with that standard interval-overlap predicate. I'll keep the window half-open ([start, end)) so a booking ending at 11:00 and another starting at 11:00 don't conflict. If the business wants "touching equals conflict," I'd change the comparisons to <=/>= accordingly.

Code:

```
import java.sql.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

class Booking {
    final int bookingId;
    final int roomId;
    final Timestamp start;
    final Timestamp end;
    final String purpose;

    Booking(int bookingId, int roomId, Timestamp start, Timestamp end, String
purpose) {
        this.bookingId = bookingId;
        this.roomId = roomId;
        this.start = start;
        this.end = end;
        this.purpose = purpose;
    }

    @Override public String toString() {
        return "Booking{id=%d, room=%d, start=%s, end=%s, purpose='%s'}"
            .formatted(bookingId, roomId, start, end, purpose);
    }
}

public class FetchOverlappingRoomBookings {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/coworking";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
```

```
        LocalDateTime reqStart = LocalDateTime.parse("2024-09-01T09:00:00");
        LocalDateTime reqEnd   = LocalDateTime.parse("2024-09-01T11:00:00");
        int roomId = 101;

        List<Booking> overlaps = findOverlaps(roomId,
                Timestamp.valueOf(reqStart),
                Timestamp.valueOf(reqEnd));

        overlaps.forEach(System.out::println);
    }

    public static List<Booking> findOverlaps(int roomId, Timestamp reqStart,
Timestamp reqEnd) {
        // Half-open window: [start, end)
        String sql = """
            SELECT booking_id, room_id, start_time, end_time, purpose
            FROM room_bookings
            WHERE room_id = ?
              AND start_time < ?
              AND end_time   > ?
            ORDER BY start_time
        """;

        List<Booking> bookings = new ArrayList<>();

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, roomId);
            ps.setTimestamp(2, reqEnd);   // existing.start < requestedEnd
            ps.setTimestamp(3, reqStart); // existing.end   > requestedStart

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    bookings.add(new Booking(
                            rs.getInt("booking_id"),
                            rs.getInt("room_id"),
                            rs.getTimestamp("start_time"),
                            rs.getTimestamp("end_time"),
                            rs.getString("purpose")
                    ));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return bookings;
    }
}
```

Reasoning to interviewer:

• The canonical overlap condition avoids the common “two OR clauses” pitfall. It’s easier to read and lets the optimizer use indexes effectively.

• I filter by room_id first to keep the scan small.

• Using [start, end) resolves boundary ambiguity; a meeting ending at 11:00 and another starting at 11:00 won’t collide. If product wants stricter rules, I’ll switch to start_time <= reqEnd and end_time >= reqStart.

• Ordering by start_time helps the UI to show conflicts chronologically.

Future scope:

• Add a unique constraint or exclusion constraint at the DB layer (MySQL can use triggers; Postgres has EXCLUDE USING GIST) to enforce no overlaps even if two app nodes race.

• Create an index on (room_id, start_time, end_time).

• Normalize to UTC in the DB and convert to user timezone in the application layer.

Scenario 3: Show recent bookings for a customer in the past 7 days

In a movie ticketing system (like BookMyShow), customers often check their recent bookings for cancellations, reprints, or loyalty points. The requirement is to fetch bookings from the last 7 days for a given customer.

Answer:

I'll calculate the cutoff date in Java (current date minus 7) and pass it as a parameter in the query. Using a PreparedStatement ensures customer_id and date are parameterized to prevent injection. I'll also return the results in descending order of booking_date so the newest tickets appear first in the UI.

Code:

```
import java.sql.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

class Booking {
    final int bookingId;
    final Date bookingDate;
    final String status;

    Booking(int bookingId, Date bookingDate, String status) {
        this.bookingId = bookingId;
        this.bookingDate = bookingDate;
        this.status = status;
    }

    @Override
    public String toString() {
        return "Booking{id=%d, date=%s, status=%s}".formatted(bookingId,
bookingDate, status);
    }
}

public class FetchRecentBookings {
    private static final String URL = "jdbc:mysql://localhost:3306/cinema";
    private static final String USER = "root";
    private static final String PASS = "password";

    public static void main(String[] args) {
        int customerId = 202;
        LocalDate cutoff = LocalDate.now().minusDays(7);

        List<Booking> bookings = getRecentBookings(customerId, cutoff);
        bookings.forEach(System.out::println);
    }
```

```
    public static List<Booking> getRecentBookings(int customerId, LocalDate cutoff)
{
        String sql = """
            SELECT booking_id, booking_date, status
            FROM bookings
            WHERE customer_id = ? AND booking_date >= ?
            ORDER BY booking_date DESC
        """;

        List<Booking> results = new ArrayList<>();
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, customerId);
            ps.setDate(2, Date.valueOf(cutoff));

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    results.add(new Booking(
                            rs.getInt("booking_id"),
                            rs.getDate("booking_date"),
                            rs.getString("status")
                    ));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return results;
    }
}
```

Reasoning to interviewer:

- LocalDate.now().minusDays(7) ensures the cutoff is dynamic, not hardcoded.
- Ordering by booking_date DESC matches user expectation—most recent bookings on top.
- Using a domain object (Booking) instead of printing in the loop allows reuse in APIs or UI layers.
- The query filters in the DB, so only relevant rows travel over the network.

Future scope:

• Add pagination if customers have many bookings.

• Show only active statuses (exclude canceled/refunded if business requires).

• Consider time zones: if booking_date includes timestamps, switch to LocalDateTime and use BETWEEN.

Scenario 4: Fetch reviews for a dish sorted by rating

In a food review portal (like Zomato or Swiggy), customers expect to see the highest-rated reviews at the top. The query must pull reviews for a dish_id, ordered by rating descending.

Answer:

I'll parameterize the dish_id and order by rating DESC. If ratings tie, I may add booking_date DESC as a secondary sort for freshness. Returning reviews as objects keeps things clean.

Code:

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

class Review {
    final int customerId;
    final int rating;
    final String review;

    Review(int customerId, int rating, String review) {
        this.customerId = customerId;
        this.rating = rating;
        this.review = review;
    }

    @Override
    public String toString() {
        return "Review{customer=%d, rating=%d, review='%s'}"
            .formatted(customerId, rating, review);
    }
}

public class FetchDishReviews {
    private static final String URL = "jdbc:mysql://localhost:3306/restaurant";
    private static final String USER = "root";
    private static final String PASS = "password";

    public static void main(String[] args) {
        int dishId = 15;
        List<Review> reviews = getDishReviews(dishId);
        reviews.forEach(System.out::println);
    }

    public static List<Review> getDishReviews(int dishId) {
        String sql = """
            SELECT customer_id, rating, review
```

```
            FROM dish_reviews
            WHERE dish_id = ?
            ORDER BY rating DESC, customer_id ASC
        """;

        List<Review> reviews = new ArrayList<>();
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, dishId);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    reviews.add(new Review(
                            rs.getInt("customer_id"),
                            rs.getInt("rating"),
                            rs.getString("review")
                    ));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return reviews;
    }
}
```

Reasoning to interviewer:

- ORDER BY rating DESC ensures the highest-rated reviews surface first, matching user expectations.
- Adding a secondary sort (e.g., review_date DESC) can make the feed stable.
- Mapping to a Review class instead of printing ensures separation of concerns.
- The PreparedStatement parameterizes dish_id, which avoids SQL injection.

Future scope:

- Add pagination (TOP 10 reviews first).
- Include customer profile info (name, avatar) with a JOIN.
- Allow filters (e.g., show only 4★ and above).

Scenario 5: Fetch transactions of a given type for a user

In a digital wallet system (like Paytm or PhonePe), users often filter their history to see only credits or only debits. We need to fetch all transactions of a specific type for a given user.

Answer:

I'll take both user_id and txn_type as parameters in the query. A PreparedStatement makes this safe and reusable. I'll return transactions in reverse chronological order, so the latest ones show first in the UI.

Code:

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

class Transaction {
    final int txnId;
    final double amount;
    final Date txnDate;

    Transaction(int txnId, double amount, Date txnDate) {
        this.txnId = txnId;
        this.amount = amount;
        this.txnDate = txnDate;
    }

    @Override
    public String toString() {
        return "Txn{id=%d, amount=%.2f, date=%s}".formatted(txnId, amount,
txnDate);
    }
}

public class FetchTransactionsByType {
    private static final String URL = "jdbc:mysql://localhost:3306/wallet";
    private static final String USER = "root";
    private static final String PASS = "password";

    public static void main(String[] args) {
        List<Transaction> txns = getTransactions(305, "credit");
        txns.forEach(System.out::println);
    }

    public static List<Transaction> getTransactions(int userId, String type) {
        String sql = """
            SELECT txn_id, amount, txn_date
            FROM wallet_transactions
```

```
            WHERE user_id = ? AND txn_type = ?
            ORDER BY txn_date DESC
        """;

        List<Transaction> transactions = new ArrayList<>();

        try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, userId);
            ps.setString(2, type);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    transactions.add(new Transaction(
                            rs.getInt("txn_id"),
                            rs.getDouble("amount"),
                            rs.getDate("txn_date")
                    ));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return transactions;
    }
}
```

Reasoning to interviewer:

- The txn_type filter ensures we only fetch credits or debits as requested.
- Sorting by txn_date DESC matches user expectations—latest first.
- By mapping results to a Transaction object, we separate concerns (querying vs displaying).
- Using PreparedStatement prevents SQL injection and allows DB query plan reuse.

Future scope:

- Add pagination if a user has thousands of transactions.
- Support date filters (last month, custom range).
- Add indexes on (user_id, txn_type, txn_date) for fast lookups.

Scenario 6: Prevent duplicate enrollments in a course

In an online course platform (like Coursera or Udemy), students should not enroll in the same course twice. Before inserting, we must check if a student is already enrolled.

Answer:

I'll query with student_id and course_id to check if a record exists. Using SELECT 1 is efficient because we don't need the full row. If the result set returns a row, the enrollment already exists.

Code:

```
import java.sql.*;

public class CheckCourseEnrollment {
    private static final String URL = "jdbc:mysql://localhost:3306/academydb";
    private static final String USER = "admin";
    private static final String PASS = "admin123";

    public static void main(String[] args) {
        int studentId = 501;
        int courseId = 1003;

        if (isAlreadyEnrolled(studentId, courseId)) {
            System.out.println("Student is already enrolled in this course.");
        } else {
            System.out.println("Student is not enrolled yet.");
        }
    }

    public static boolean isAlreadyEnrolled(int studentId, int courseId) {
        String sql = "SELECT 1 FROM course_enrollments WHERE student_id = ?
AND course_id = ?";

        try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, studentId);
            ps.setInt(2, courseId);

            try (ResultSet rs = ps.executeQuery()) {
                return rs.next(); // true if enrollment exists
            }
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }
```

```
}
```

Reasoning to interviewer:

- Using SELECT 1 avoids unnecessary column fetch—lighter on DB.
- The query checks existence in O(1) if (student_id, course_id) has a composite index.
- Wrapping logic in a method makes it reusable in both insert validation and API checks.
- Returning a boolean simplifies integration with enrollment service logic.

Future scope:

- Add a unique constraint on (student_id, course_id) at the DB level to guarantee safety even under concurrent requests.
- Introduce proper exception handling for concurrent enrollment attempts.
- Extend logic to track enrollment status (active, completed, dropped).

Scenario 7: Calculate average restaurant rating

In a food delivery app (like Swiggy/Zomato), customers want to see the overall average rating of a restaurant before ordering. The query must calculate the mean of all submitted ratings.

Answer:

I'll use SQL's AVG() aggregate function, which calculates the average directly in the database without fetching all rows into Java. Using a PreparedStatement ensures safety and reusability. I'll also check for NULL, because if no reviews exist, AVG() will return NULL.

Code:

```
import java.sql.*;

public class RestaurantRatingCalculator {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/foodapp";
    private static final String DB_USER = "admin";
    private static final String DB_PASS = "admin123";

    public static void main(String[] args) {
        int restaurantId = 204;
        Double avgRating = calculateAverageRating(restaurantId);

        if (avgRating == null) {
            System.out.println("No reviews available yet for restaurant " +
restaurantId);
        } else {
            System.out.println("Average rating: " + avgRating);
        }
    }

    public static Double calculateAverageRating(int restaurantId) {
        String sql = "SELECT AVG(rating) AS avg_score FROM restaurant_reviews
WHERE restaurant_id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, restaurantId);

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    double avg = rs.getDouble("avg_score");
                    return rs.wasNull() ? null : avg;
                }
            }
```

```
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

Reasoning to interviewer:

- AVG(rating) lets the DB handle aggregation efficiently.
- Handling NULL is important—restaurants with no reviews should show “No reviews yet” instead of 0.0.
- Using PreparedStatement makes the code safe from SQL injection and easy to reuse.
- By returning Double, I can distinguish between no result (null) and actual numeric average.

Future scope:

- Add a secondary metric like total number of ratings (COUNT).
- Cache popular restaurant averages to avoid recalculating for every request.
- Use rounding (e.g., one decimal place) for consistent UI display.

Scenario 8: Prevent duplicate seat bookings

In a cinema booking system (like BookMyShow), a customer should not book two seats for the same show. Before inserting a reservation, we need to check if one already exists.

Answer:

I'll query with customer_id and show_id to check if a record exists. Using SELECT 1 is efficient because we don't need the full row. If the ResultSet has at least one record, it means the customer already booked for that show.

Code:

```
import java.sql.*;

public class SeatReservationChecker {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/cinemadb";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "root123";

    public static void main(String[] args) {
        int customerId = 301;
        int showId = 9002;

        if (hasExistingReservation(customerId, showId)) {
            System.out.println("Customer already has a booking for this show.");
        } else {
            System.out.println("Customer has no booking for this show.");
        }
    }

    public static boolean hasExistingReservation(int customerId, int showId) {
        String sql = "SELECT 1 FROM seat_reservations WHERE customer_id = ?
AND show_id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, customerId);
            ps.setInt(2, showId);

            try (ResultSet rs = ps.executeQuery()) {
                return rs.next(); // true if a record exists
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return false;
```

```
    }
}
```

Reasoning to interviewer:

- SELECT 1 avoids fetching unnecessary columns—faster and lighter.
- Checking existence in the DB is critical before insert, but I'd also enforce uniqueness at the DB level (unique constraint).
- Wrapping logic in a reusable method (hasExistingReservation) makes it easy to call from service/business logic.
- PreparedStatement keeps things secure from injection.

Future scope:

- Add a unique constraint on (customer_id, show_id) in the DB to guarantee no duplicates even if multiple app servers run in parallel.
- Extend the query to also check if the booking is "active" (ignore canceled).
- If concurrency is high, use a transaction with proper isolation to avoid race conditions.

Scenario 9: Fetch failed payments of a customer

In an e-commerce system (like Flipkart or Amazon), customer support often needs to check all failed transactions for troubleshooting or refunds. The task is to retrieve all failed payments for a given customer.

Answer:

I'll use a PreparedStatement to filter by both customer_id and status = 'failed'. This ensures only failed transactions of that customer are returned. I'll also sort results by date, so support teams can see the most recent failures first.

Code:

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

class Payment {
    final int paymentId;
    final double amount;
    final Date date;

    Payment(int paymentId, double amount, Date date) {
        this.paymentId = paymentId;
        this.amount = amount;
        this.date = date;
    }

    @Override
    public String toString() {
        return "Payment{id=%d, amount=%.2f, date=%s}"
                .formatted(paymentId, amount, date);
    }
}

public class FailedPaymentsFetcher {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/ecommerce";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        List<Payment> failed = getFailedPayments(110);
        failed.forEach(System.out::println);
    }

    public static List<Payment> getFailedPayments(int customerId) {
        String sql = """
            SELECT payment_id, amount, payment_date
            FROM order_payments
```

```
            WHERE customer_id = ? AND status = ?
            ORDER BY payment_date DESC
        """;

        List<Payment> list = new ArrayList<>();

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, customerId);
            ps.setString(2, "failed");

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    list.add(new Payment(
                            rs.getInt("payment_id"),
                            rs.getDouble("amount"),
                            rs.getDate("payment_date")
                    ));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return list;
    }
}
```

Reasoning to interviewer:

- Filtering on both customer_id and status ensures only the relevant rows are fetched.
- Using ORDER BY payment_date DESC makes debugging easier by showing the latest failures first.
- Returning a domain object (Payment) instead of just printing makes the method reusable in APIs and services.
- PreparedStatement keeps the query safe and allows DB query plan reuse.

Future scope:

- Add pagination for customers with long histories of failed attempts.
- Include failure reason in the query for better support insights.
- Create an index on (customer_id, status, payment_date) for faster lookups.

Scenario 10: Insert new library member securely

In a library system, when a user registers, their details must be added to library_members. To prevent SQL injection and ensure efficiency, PreparedStatement is the right choice.

Answer:

I'll prepare an INSERT query with placeholders for name, membership type, and email. Then bind parameters securely with setString. Using executeUpdate will insert the record and return the number of rows affected.

Code:

```
import java.sql.*;

public class AddLibraryMember {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/librarydb";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "root123";

    public static void main(String[] args) {
        addMember("Alice Johnson", "Premium", "alice.j@example.com");
    }

    public static void addMember(String fullName, String membershipType, String
email) {
        String sql = "INSERT INTO library_members (full_name, membership_type,
email) VALUES (?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, fullName);
            ps.setString(2, membershipType);
            ps.setString(3, email);

            int rows = ps.executeUpdate();
            if (rows > 0) {
                System.out.println("New member added successfully!");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- PreparedStatement prevents SQL injection by separating SQL from user input.
- Using placeholders ? keeps the query reusable for multiple inserts.
- executeUpdate is efficient for INSERT/UPDATE/DELETE since it returns the affected row count.
- Wrapping this in a reusable method makes it easier to integrate with registration logic.

Future scope:

- Enforce unique constraint on email to prevent duplicate registrations.
- Return the auto-generated member ID using RETURN_GENERATED_KEYS.
- Batch inserts if importing multiple members at once for efficiency.

Scenario 11: Atomic wallet fund transfer with transactions

In a mobile wallet app (like Paytm or Google Pay), fund transfers must be atomic. If the sender's account is debited but the receiver's credit fails, it would cause inconsistencies. We need JDBC transaction management so that either both updates succeed or both are rolled back.

Answer:

I'll disable auto-commit, run both updates (debit sender, credit receiver), and commit only if both succeed. If any step fails, I'll roll back to maintain data consistency.

Code:

```
import java.sql.*;

public class WalletFundTransfer {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/walletdb";
    private static final String DB_USER = "admin";
    private static final String DB_PASS = "admin123";

    public static void main(String[] args) {
        transferFunds(1001, 1005, 250.00);
    }

    public static void transferFunds(int senderId, int receiverId, double amount) {
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS)) {
            conn.setAutoCommit(false);

            try {
                String deductQuery = "UPDATE wallet_accounts SET balance = balance
- ? WHERE account_id = ?";
                try (PreparedStatement deductStmt =
conn.prepareStatement(deductQuery)) {
                    deductStmt.setDouble(1, amount);
                    deductStmt.setInt(2, senderId);
                    deductStmt.executeUpdate();
                }

                String addQuery = "UPDATE wallet_accounts SET balance = balance +
? WHERE account_id = ?";
                try (PreparedStatement addStmt = conn.prepareStatement(addQuery)) {
                    addStmt.setDouble(1, amount);
                    addStmt.setInt(2, receiverId);
                    addStmt.executeUpdate();
                }

                conn.commit();
                System.out.println("Transfer completed successfully!");
```

```
            } catch (SQLException ex) {
                conn.rollback();
                System.out.println("Transfer failed. Transaction rolled back.");
                ex.printStackTrace();
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- setAutoCommit(false) ensures multiple SQL statements are grouped into one transaction.
- If either debit or credit fails, rollback keeps the system consistent—no partial updates.
- Using PreparedStatement makes the queries safe and parameterized.
- Committing only after both updates succeed implements the all-or-nothing principle (ACID).

Future scope:

- Add balance validation before deduction (check sufficient funds).
- Use transaction isolation level (e.g., READ_COMMITTED) to prevent dirty reads.
- Consider optimistic locking (version column) to avoid race conditions in concurrent transfers.

Scenario 12: Display all travelers in tabular format

In a travel booking app, admins often need to view all registered travelers. The trip_users table stores details like name, email, and phone. We must fetch and display them neatly.

Answer:

I'll run a simple SELECT query, iterate over the ResultSet, and print data in tabular format using printf for clean alignment.

Code:

```
import java.sql.*;

public class FetchTravelers {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/travel_agency";
    private static final String DB_USER = "admin";
    private static final String DB_PASS = "pass123";

    public static void main(String[] args) {
        listTravelers();
    }

    public static void listTravelers() {
        String sql = "SELECT traveler_name, email, phone FROM trip_users";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {

            System.out.printf("%-20s %-30s %-15s%n", "Name", "Email", "Phone");
            System.out.println("-----------------------------------------------------------------");

            while (rs.next()) {
                String name = rs.getString("traveler_name");
                String email = rs.getString("email");
                String phone = rs.getString("phone");

                System.out.printf("%-20s %-30s %-15s%n", name, email, phone);
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Using Statement here is fine since the query has no user input, but for dynamic filters I'd use PreparedStatement.

• printf formatting ensures the output is neatly tabular, improving readability in console reports.

• try-with-resources automatically closes ResultSet, Statement, and Connection.

• This approach separates data fetching from formatting, making it easy to adapt for other UIs.

Future scope:

• Allow sorting (e.g., by name or registration date).

• Add filtering (like list travelers by country).

• Integrate with a reporting library for export to CSV/Excel.

Scenario 13: Update inventory stock after book purchase

In a bookstore system (like Amazon Books), whenever a customer completes a purchase, the available stock must be decremented to reflect the sale. This update needs to be safe and efficient.

Answer:

I'll use a parameterized UPDATE query with PreparedStatement to reduce the stock count of the purchased book. PreparedStatement ensures security against SQL injection and improves performance. I'll also check how many rows were updated to confirm that the book exists.

Code:

```
import java.sql.*;

public class UpdateBookInventory {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/bookstore";
    private static final String DB_USER = "admin";
    private static final String DB_PASS = "admin123";

    public static void main(String[] args) {
        updateStock(501, 2); // Book ID 501, reduce stock by 2
    }

    public static void updateStock(int bookId, int qty) {
        String sql = "UPDATE inventory SET stock = stock - ? WHERE book_id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, qty);
            ps.setInt(2, bookId);

            int rowsAffected = ps.executeUpdate();
            if (rowsAffected > 0) {
                System.out.println("Stock updated successfully.");
            } else {
                System.out.println("Book not found in inventory.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Using stock = stock - ? performs the decrement atomically inside the database.

• PreparedStatement avoids SQL injection and can reuse the compiled query.

• Checking rowsAffected helps handle cases where the book doesn't exist in inventory.

• try-with-resources ensures that connections and statements close automatically.

Future scope:

• Add a condition like AND stock >= ? to prevent overselling when stock is low.

• Wrap updates in a transaction when multiple items are purchased together.

• Implement optimistic locking (version field) to handle concurrency in high-traffic systems.

Scenario 14: Update tenant's contact details

In a property rental system, when a tenant changes their email or phone, the admin must securely update their contact details in the database.

Answer:

I'll use a parameterized UPDATE query with PreparedStatement to modify both email and phone_number for the tenant. The tenant_id uniquely identifies which record to update. The return value from executeUpdate indicates success or failure.

Code:

```
import java.sql.*;

public class UpdateTenantContact {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/rental_system";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "root123";

    public static void main(String[] args) {
        updateContact(2002, "tenant.new@example.com", "9876543210");
    }

    public static void updateContact(int tenantId, String newEmail, String
newPhone) {
        String sql = "UPDATE tenants SET email = ?, phone_number = ? WHERE
tenant_id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, newEmail);
            ps.setString(2, newPhone);
            ps.setInt(3, tenantId);

            int rowsUpdated = ps.executeUpdate();
            if (rowsUpdated > 0) {
                System.out.println("Tenant contact details updated successfully.");
            } else {
                System.out.println("Tenant record not found.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- PreparedStatement prevents SQL injection by separating parameters from SQL logic.
- Updating multiple fields in one statement ensures atomicity of the change.
- Returning rowsUpdated helps confirm that the tenant exists before claiming success.
- try-with-resources handles automatic cleanup of database resources.

Future scope:

- Add validation for email format and phone number before updating.
- Track update timestamps with an updated_at column.
- Allow partial updates (e.g., only phone or only email) by adjusting query logic.

Scenario 15: Check event ticket availability

In a concert booking system, every event has a seats_left column. Before allowing a booking, we must check whether seats are still available.

Answer:

I'll use a PreparedStatement to fetch seats_left for the given event_id. If the value is greater than zero, tickets can be booked. If no rows are returned, the event doesn't exist.

Code:

```
import java.sql.*;

public class EventAvailabilityCheck {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/concertdb";
    private static final String DB_USER = "admin";
    private static final String DB_PASS = "admin123";

    public static void main(String[] args) {
        checkAvailability(205); // Example event ID
    }

    public static void checkAvailability(int eventId) {
        String sql = "SELECT seats_left FROM events WHERE event_id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, eventId);

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    int remainingSeats = rs.getInt("seats_left");
                    if (remainingSeats > 0) {
                        System.out.println("Tickets are available for booking.");
                    } else {
                        System.out.println("No tickets available for booking.");
                    }
                } else {
                    System.out.println("Event not found.");
                }
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
```

```
}
```

Reasoning to interviewer:

- PreparedStatement ensures safety and allows dynamic binding of event_id.
- Checking seats_left > 0 ensures we only allow booking when capacity exists.
- Handling the else case (no rows) avoids false assumptions when the event ID is invalid.
- Wrapping in a method makes it reusable across multiple parts of the booking flow.

Future scope:

- Use transactions if checking availability is followed by seat reservation to avoid overselling.
- Introduce optimistic locking or seats_left >= ? condition to prevent race conditions.
- Add monitoring (e.g., alerts when seats_left is low) to trigger marketing push.

Scenario 16: Insert a product review securely

In an e-commerce app, customers post reviews with a star rating and text feedback. These reviews go into a customer_reviews table.

Answer:

I'll use a PreparedStatement to insert product_id, rating, and review_text. This protects against SQL injection and allows clean parameter binding. The executeUpdate call performs the insert.

Code:

```
import java.sql.*;

public class InsertProductReview {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/shopdb";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "rootpass";

    public static void main(String[] args) {
        addReview(3001, 4, "Good quality and fast delivery!");
    }

    public static void addReview(int productId, int rating, String review) {
        String sql = "INSERT INTO customer_reviews (product_id, rating,
review_text) VALUES (?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, productId);
            ps.setInt(2, rating);
            ps.setString(3, review);

            int rows = ps.executeUpdate();
            if (rows > 0) {
                System.out.println("Review added successfully.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• Using PreparedStatement avoids SQL injection and is efficient for repeated inserts.

• Binding parameters with setInt and setString ensures type safety.

• Checking rows > 0 confirms the insert worked.

• Wrapping the insert logic in a method allows reuse for APIs or batch processing.

Future scope:

• Return the generated review ID using RETURN_GENERATED_KEYS.

• Add validation (rating should be 1–5, review length limit).

• Support bulk insert for importing reviews from external sources.

Scenario 17: Insert a candidate's job application

A hiring portal needs to store each application (name, phone, position) safely and repeatedly. We should use a parameterized insert so user input never touches SQL directly.

Answer:

I'll prepare an INSERT with placeholders for name, phone, and position, bind the values with a PreparedStatement, and execute it. For real systems I'd also fetch the generated application ID so I can show a confirmation or track status later.

Code:

```
import java.sql.*;

public class JobApplicationInsert {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/hrdb";
    private static final String DB_USER = "hr_user";
    private static final String DB_PASS = "hrpass";

    public static void main(String[] args) {
        int appId = submitApplication("Michael Smith", "+1-555-123-4567", "Data
Analyst");
        if (appId > 0) {
            System.out.println("Application submitted successfully. ID: " + appId);
        } else {
            System.out.println("Application submission failed.");
        }
    }

    public static int submitApplication(String fullName, String phone, String position)
{
        String sql = "INSERT INTO applications (full_name, phone, position) VALUES
(?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {

            ps.setString(1, fullName);
            ps.setString(2, phone);
            ps.setString(3, position);

            int rows = ps.executeUpdate();
            if (rows > 0) {
                try (ResultSet keys = ps.getGeneratedKeys()) {
                    if (keys.next()) return keys.getInt(1);
                }
```

```
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return -1;
    }
}
```

Reasoning to interviewer:

- PreparedStatement prevents SQL injection and lets the DB reuse the plan.
- Returning the generated key is practical—you usually need an application ID for downstream flows (email, status page).
- Wrapping in a method keeps the main clean and makes this easy to plug into a service/controller.

Future scope:

- Add basic validation (phone format, non-empty name).
- Enforce uniqueness on (full_name, phone, position) if the business wants to block duplicates.
- Capture timestamps (created_at) and source channel (web/mobile) for analytics.

Scenario 18: Delete a user and their purchases atomically

A subscription app must permanently remove a user and every purchase tied to them. If either delete fails, nothing should change. This is a classic all-or-nothing transaction.

Answer:

I'll start a transaction, delete child rows (purchases) first to satisfy FK constraints, then delete the user row. If both succeed, commit; if anything breaks, roll back. In production, I'd prefer an FK with ON DELETE CASCADE to simplify logic, but I'll still use a transaction for safety.

Code:

```
import java.sql.*;

public class DeleteUserAndPurchases {
    private static final String DB_URL =
"jdbc:mysql://localhost:3306/subscriptiondb";
    private static final String DB_USER = "service_admin";
    private static final String DB_PASS = "adminpass";

    public static void main(String[] args) {
        boolean ok = deleteUserCompletely(5001);
        System.out.println(ok ? "User and their purchases deleted successfully." :
"Delete failed.");
    }

    public static boolean deleteUserCompletely(int userId) {
        String deletePurchases = "DELETE FROM purchases WHERE user_id = ?";
        String deleteUser      = "DELETE FROM users WHERE user_id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS)) {
            conn.setAutoCommit(false);
            try (PreparedStatement ps1 = conn.prepareStatement(deletePurchases);
                 PreparedStatement ps2 = conn.prepareStatement(deleteUser)) {

                ps1.setInt(1, userId);
                ps1.executeUpdate();           // delete child rows first

                ps2.setInt(1, userId);
                int userRows = ps2.executeUpdate(); // then parent row

                if (userRows == 0) {
                    // user did not exist; decide business behavior
                    conn.rollback();
                    return false;
                }
```

```
            conn.commit();
            return true;
        } catch (SQLException ex) {
            conn.rollback();
            ex.printStackTrace();
            return false;
        }
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
  }
}
```

Reasoning to interviewer:

- Transactions (setAutoCommit(false)) guarantee atomicity—either both deletes commit or neither does.
- Deleting children before parent avoids FK violations when cascade isn't configured.
- Checking userRows lets us handle "user not found" explicitly instead of silently "succeeding."
- Using two prepared statements keeps the code safe and readable.

Future scope:

- Prefer an FK with ON DELETE CASCADE and just delete from users—the DB cleans up purchases.
- Add soft-delete (status flags) if compliance requires retention; hard-delete only after a grace window.
- Log deletions to an audit table inside the same transaction for traceability.

Scenario 19: Retrieve a user's reading history

In a digital library, users should be able to see all the books they've borrowed in the past. The reading_history table stores borrow and return dates for each user.

Answer:

I'll write a parameterized SELECT query that filters by user_id. Using PreparedStatement keeps it secure, and looping through the ResultSet prints each borrowed book with borrow and return dates.

Code:

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

class ReadingRecord {
    final int bookId;
    final Date borrowDate;
    final Date returnDate;

    ReadingRecord(int bookId, Date borrowDate, Date returnDate) {
        this.bookId = bookId;
        this.borrowDate = borrowDate;
        this.returnDate = returnDate;
    }

    @Override
    public String toString() {
        return "Book ID: " + bookId + ", Borrowed: " + borrowDate + ", Returned: " +
returnDate;
    }
}

public class RetrieveReadingHistory {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/librarydb";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        List<ReadingRecord> history = getReadingHistory(42);
        history.forEach(System.out::println);
    }

    public static List<ReadingRecord> getReadingHistory(int userId) {
        String sql = "SELECT book_id, borrow_date, return_date FROM
reading_history WHERE user_id = ?";
        List<ReadingRecord> records = new ArrayList<>();
```

```
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, userId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    records.add(new ReadingRecord(
                            rs.getInt("book_id"),
                            rs.getDate("borrow_date"),
                            rs.getDate("return_date")
                    ));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return records;
    }
}
```

Reasoning to interviewer:

- PreparedStatement ensures only one user's history is fetched securely.
- The domain class ReadingRecord keeps results structured and reusable beyond console output.
- Iterating the ResultSet ensures all past borrowings are retrieved.
- Using try-with-resources guarantees automatic cleanup.

Future scope:

- Join with the books table to display book titles instead of just IDs.
- Add ordering by borrow_date DESC to show recent history first.
- Allow filters (e.g., show only currently borrowed books where return_date IS NULL).

Scenario 20: Retrieve a user's flight bookings

In a travel platform, admins need to review the complete flight booking history of a customer. The flight_bookings table stores destination and booking_time for each user.

Answer:

I'll prepare a SELECT query that retrieves all rows for a given user_id. A PreparedStatement ensures safety, and the ResultSet iteration prints destination and booking time for each record.

Code:

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

class FlightBooking {
    final String destination;
    final Timestamp bookingTime;

    FlightBooking(String destination, Timestamp bookingTime) {
        this.destination = destination;
        this.bookingTime = bookingTime;
    }

    @Override
    public String toString() {
        return "Destination: " + destination + ", Booking Time: " + bookingTime;
    }
}

public class RetrieveFlightBookings {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/travelhub";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        List<FlightBooking> bookings = getBookingsByUser(7);
        bookings.forEach(System.out::println);
    }

    public static List<FlightBooking> getBookingsByUser(int userId) {
        String sql = "SELECT destination, booking_time FROM flight_bookings
WHERE user_id = ?";
        List<FlightBooking> list = new ArrayList<>();

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
```

```
            PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, userId);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    list.add(new FlightBooking(
                            rs.getString("destination"),
                            rs.getTimestamp("booking_time")
                    ));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return list;
    }
}
```

Reasoning to interviewer:

- The query retrieves only the bookings for the given user, keeping the dataset small and relevant.
- Mapping to a FlightBooking object makes the data reusable (e.g., API response).
- Using Timestamp captures both date and time, which is critical for flight logs.
- try-with-resources ensures resource cleanup.

Future scope:

- Add ordering (ORDER BY booking_time DESC) for latest-first display.
- Join with flights table to show airline and flight number along with destination.
- Provide filters (e.g., bookings within the last 6 months).

Scenario 21: Fetch reservations for a user within a date range

In a hotel booking system, customers often want to see their reservations between two dates (for example, all summer bookings). The reservations table contains check-in, check-out, and price details.

Answer:

I'll prepare a query filtering by user_id and use BETWEEN ? AND ? on check_in_date to retrieve all relevant reservations. A PreparedStatement ensures secure parameter binding for both user ID and date range.

Code:

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

class Reservation {
    final int reservationId;
    final Date checkIn;
    final Date checkOut;
    final double totalPrice;

    Reservation(int reservationId, Date checkIn, Date checkOut, double totalPrice) {
        this.reservationId = reservationId;
        this.checkIn = checkIn;
        this.checkOut = checkOut;
        this.totalPrice = totalPrice;
    }

    @Override
    public String toString() {
        return "Reservation ID: " + reservationId + ", Check-in: " + checkIn +
                ", Check-out: " + checkOut + ", Price: " + totalPrice;
    }
}

public class RetrieveReservations {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/hotelbooking";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        List<Reservation> reservations = getReservations(15,
                Date.valueOf("2024-05-01"), Date.valueOf("2024-08-31"));
        reservations.forEach(System.out::println);
    }
```

```
    public static List<Reservation> getReservations(int userId, Date start, Date end)
{
        String sql = """
            SELECT reservation_id, check_in_date, check_out_date, total_price
            FROM reservations
            WHERE user_id = ? AND check_in_date BETWEEN ? AND ?
        """;

        List<Reservation> list = new ArrayList<>();

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, userId);
            ps.setDate(2, start);
            ps.setDate(3, end);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    list.add(new Reservation(
                            rs.getInt("reservation_id"),
                            rs.getDate("check_in_date"),
                            rs.getDate("check_out_date"),
                            rs.getDouble("total_price")
                    ));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return list;
    }
}
```

Reasoning to interviewer:

- BETWEEN ? AND ? makes the date filter concise and efficient.
- PreparedStatement protects against SQL injection and allows dynamic user/date input.
- Wrapping results in a Reservation object keeps the code structured and reusable.
- This approach can be easily extended for UI or reporting needs.

Future scope:

- Allow filtering by check-out date as well, depending on the use case.
- Add sorting (e.g., by check-in date descending).
- Support pagination if users have many bookings.

Scenario 22: Calculate total expenses in the last 30 days

In an expense tracker app, a user should be able to see how much they spent in the past 30 days. The expenses table holds all records.

Answer:

I'll use SQL's SUM(amount) function to calculate the total. I'll dynamically compute the cutoff date in Java (LocalDate.now().minusDays(30)) and pass it into the query.

Code:

```
import java.sql.*;
import java.time.LocalDate;

public class CalculateMonthlyExpenses {
    private static final String DB_URL =
"jdbc:mysql://localhost:3306/expensetracker";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        double total = getTotalExpenses(9);
        System.out.println("Total spent in last 30 days: " + total);
    }

    public static double getTotalExpenses(int userId) {
        String sql = "SELECT SUM(amount) AS total_spent FROM expenses " +
                     "WHERE user_id = ? AND expense_date >= ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, userId);
            ps.setDate(2, Date.valueOf(LocalDate.now().minusDays(30)));

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return rs.getDouble("total_spent");
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return 0.0;
    }
}
```

Reasoning to interviewer:

- SUM(amount) leverages the database for aggregation instead of summing in Java.
- Using LocalDate.now().minusDays(30) ensures the cutoff date is dynamic.
- Returning a double makes it easy to plug into reports or dashboards.
- PreparedStatement keeps queries safe and efficient.

Future scope:

- Add grouping by category to show category-wise spending.
- Allow configurable date ranges (weekly, monthly, yearly).
- Handle cases where no rows exist (return 0 gracefully).

Scenario 23: Update ticket count after booking

In an event booking platform, each booking reduces the number of available tickets for that event. The events table holds an available_tickets column that must be decremented every time a customer books a seat.

Answer:

I'll use a parameterized UPDATE query to subtract tickets from the available count for the specific event_id. After execution, I'll check rowsUpdated to confirm whether the event existed and the stock was updated.

Code:

```
import java.sql.*;

public class UpdateTicketCountExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        updateTicketCount(202, 1); // Example event ID and decrement by 1
    }

    public static void updateTicketCount(int eventId, int quantity) {
        String sql = "UPDATE events SET available_tickets = available_tickets - ?
WHERE event_id = ?";

        try (Connection connection = DriverManager.getConnection(DB_URL,
DB_USER, DB_PASS);
             PreparedStatement ps = connection.prepareStatement(sql)) {

            ps.setInt(1, quantity);
            ps.setInt(2, eventId);

            int rowsUpdated = ps.executeUpdate();
            if (rowsUpdated > 0) {
                System.out.println("Ticket count updated successfully.");
            } else {
                System.out.println("Event not found.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• The update happens directly in the database (available_tickets = available_tickets - ?), ensuring atomicity.

• PreparedStatement ensures safe parameter binding and avoids SQL injection.

• Checking the number of rows affected verifies whether the event exists.

• Encapsulating logic in a method makes it reusable in booking workflows.

Future scope:

• Add a condition AND available_tickets >= ? to prevent overselling.

• Wrap booking + ticket update in a transaction to maintain atomicity.

• Implement optimistic locking (version field) to handle concurrency in high-traffic scenarios.

Scenario 24: Retrieve low-stock products for restocking

In an online store, inventory managers need a quick report of products running low on stock. The products table contains stock levels, and any product below a given threshold should be flagged.

Answer:

I'll prepare a query with a stock_quantity < ? condition and fetch product details. A PreparedStatement ensures security. Iterating through the ResultSet prints all products below the threshold.

Code:

```
import java.sql.*;

public class RetrieveLowStockProductsExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        getLowStockProducts(10); // Example threshold
    }

    public static void getLowStockProducts(int threshold) {
        String sql = "SELECT product_id, name, price, stock_quantity FROM
products WHERE stock_quantity < ?";

        try (Connection connection = DriverManager.getConnection(DB_URL,
DB_USER, DB_PASS);
             PreparedStatement ps = connection.prepareStatement(sql)) {

            ps.setInt(1, threshold);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    int productId = rs.getInt("product_id");
                    String productName = rs.getString("name");
                    double price = rs.getDouble("price");
                    int stockQuantity = rs.getInt("stock_quantity");

                    System.out.println("Product ID: " + productId +
                            ", Name: " + productName +
                            ", Price: " + price +
                            ", Stock: " + stockQuantity);
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
```

```
        }
    }
}
```

Reasoning to interviewer:

- The parameterized query prevents SQL injection and allows different thresholds dynamically.
- The output provides clear product details (ID, name, price, stock), making it useful for inventory reports.
- Encapsulating logic in a method lets us reuse it in scheduled jobs or dashboards.
- try-with-resources ensures connections, statements, and result sets close cleanly.

Future scope:

- Sort results by stock_quantity ASC so the most urgent items appear first.
- Export results to CSV/Excel for business teams.
- Trigger automated purchase orders when stock falls below a critical level.

Scenario 25: Check if a user's subscription is active

In a subscription platform (like Netflix or Spotify), each user's plan has a start date and end date. To validate access, the system must check if today's date falls before or on the subscription end date.

Answer:

I'll query the subscriptions table with a condition end_date >= currentDate. If a row exists, the subscription is active. Using PreparedStatement allows me to safely pass the user ID and current date.

Code:

```
import java.sql.*;
import java.time.LocalDate;

public class CheckSubscriptionActiveExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        checkSubscription(101);
    }

    public static void checkSubscription(int userId) {
        String sql = "SELECT end_date FROM subscriptions WHERE user_id = ?
AND end_date >= ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, userId);
            ps.setDate(2, Date.valueOf(LocalDate.now()));

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    Date endDate = rs.getDate("end_date");
                    System.out.println("Subscription is active. End date: " + endDate);
                } else {
                    System.out.println("No active subscription found.");
                }
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
```

```
}
```

Reasoning to interviewer:

- Filtering on end_date >= currentDate ensures only currently valid subscriptions are retrieved.
- PreparedStatement secures the query and allows reuse with different users.
- Returning the end_date is useful for notifying users when their plan expires.
- Encapsulating the logic in a method keeps the code modular and testable.

Future scope:

- Add start_date <= currentDate in the query to ensure future subscriptions aren’t marked active.
- Handle multiple subscriptions (if a user can have overlapping plans).
- Return plan details (plan type, price) for richer UI feedback.

Scenario 26: Retrieve available books in a library

In a library management system, only books marked as available should be shown for borrowing. The books table stores the status in a boolean field available.

Answer:

I'll query all rows where available = true. Using PreparedStatement ensures safety, and looping through the ResultSet prints each available book.

Code:

```
import java.sql.*;

public class RetrieveAvailableBooksExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        listAvailableBooks();
    }

    public static void listAvailableBooks() {
        String sql = "SELECT book_id, title, author FROM books WHERE available =
true";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                int bookId = rs.getInt("book_id");
                String title = rs.getString("title");
                String author = rs.getString("author");

                System.out.println("Book ID: " + bookId +
                                   ", Title: " + title +
                                   ", Author: " + author);
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- The WHERE available = true condition ensures only borrowable books are retrieved.
- PreparedStatement is used for consistency and future extension (e.g., filtering by author or category).
- Using try-with-resources cleans up Connection, Statement, and ResultSet automatically.
- Printing structured details makes the list usable directly in console-based reports.

Future scope:

- Sort results by title or author for better user experience.
- Add pagination if the library has thousands of books.
- Extend query to also fetch genre, publication year, or shelf location.

Scenario 27: Update order status to "Pending"

In a restaurant ordering system, when a customer places an order, the system should mark its status as "Pending" until the restaurant confirms or rejects it.

Answer:

I'll use a parameterized UPDATE query on the orders table. With PreparedStatement, I can bind both the new status and the order ID. After execution, I'll check if a row was updated to confirm success.

Code:

```
import java.sql.*;

public class UpdateOrderStatusExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        updateOrderStatus(101, "Pending"); // Example order ID
    }

    public static void updateOrderStatus(int orderId, String status) {
        String sql = "UPDATE orders SET order_status = ? WHERE order_id = ?";

        try (Connection connection = DriverManager.getConnection(DB_URL,
DB_USER, DB_PASS);
             PreparedStatement ps = connection.prepareStatement(sql)) {

            ps.setString(1, status);
            ps.setInt(2, orderId);

            int rowsUpdated = ps.executeUpdate();
            if (rowsUpdated > 0) {
                System.out.println("Order status updated to " + status + ".");
            } else {
                System.out.println("Order not found.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• PreparedStatement makes the query secure and parameterized.

• Updating the status directly in the database ensures data integrity and prevents race conditions with concurrent processes.

• Checking rowsUpdated validates that the order ID actually exists.

• Encapsulating logic in a method allows flexibility (e.g., reuse for status transitions like "Confirmed" or "Delivered").

Future scope:

• Add transaction handling if order updates need to be coupled with inventory updates.

• Track status history in a separate audit table for analytics.

• Enforce valid statuses with ENUM or a lookup table.

Scenario 28: Update loyalty points after a purchase

In an e-commerce loyalty program, users earn points for each purchase. The system must update their points balance by incrementing the existing value.

Answer:

I'll use an UPDATE query with points = points + ? to atomically increase the balance. PreparedStatement allows secure binding of the earned points and user ID.

Code:

```
import java.sql.*;

public class UpdateLoyaltyPointsExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        addPoints(101, 100); // Example user ID and earned points
    }

    public static void addPoints(int userId, int points) {
        String sql = "UPDATE loyalty_points SET points = points + ? WHERE user_id
= ?";

        try (Connection connection = DriverManager.getConnection(DB_URL,
DB_USER, DB_PASS);
             PreparedStatement ps = connection.prepareStatement(sql)) {

            ps.setInt(1, points);
            ps.setInt(2, userId);

            int rowsUpdated = ps.executeUpdate();
            if (rowsUpdated > 0) {
                System.out.println("Loyalty points updated successfully.");
            } else {
                System.out.println("User not found.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• points = points + ? ensures atomic updates, avoiding race conditions when multiple purchases happen.

• PreparedStatement prevents SQL injection and optimizes repeated executions.

• Checking rowsUpdated helps confirm whether the user exists before claiming success.

• Wrapping logic in a method makes it reusable in purchase workflows.

Future scope:

• Implement transactions if loyalty updates need to be tied to order confirmation.

• Add a cap or tier system to handle maximum loyalty points.

• Track point history in a separate table for auditing and customer support.

Scenario 29: Insert new product feedback

In a marketplace, buyers leave reviews after purchasing items. The product_feedback table stores the item ID, buyer ID, score, and comment.

Answer:

I'll use a parameterized INSERT query with PreparedStatement to safely insert a feedback row. I'll also validate the score before insertion since it must fall between 1 and 5.

Code:

```
import java.sql.*;

public class InsertFeedback {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/appdb";
    private static final String DB_USER = "appuser";
    private static final String DB_PASS = "apppass";

    public static void main(String[] args) {
        insertFeedback(101, 202, 5, "Excellent quality and fast shipping!");
    }

    public static void insertFeedback(int itemId, int buyerId, int score, String
comment) {
        if (score < 1 || score > 5) {
            throw new IllegalArgumentException("Score must be between 1 and 5");
        }

        String sql = "INSERT INTO product_feedback (item_id, buyer_id, score,
comment) VALUES (?, ?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, itemId);
            ps.setInt(2, buyerId);
            ps.setInt(3, score);
            ps.setString(4, comment.trim());

            int rows = ps.executeUpdate();
            if (rows > 0) {
                System.out.println("Feedback inserted successfully.");
            } else {
                System.out.println("No row inserted.");
            }

        } catch (SQLException e) {
```

```
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- PreparedStatement ensures the insert is secure against SQL injection.
- Input validation (score range, trimmed comment) enforces business rules before persisting.
- Checking the row count from executeUpdate confirms success.
- try-with-resources guarantees all resources are closed automatically.

Future scope:

- Return the generated feedback ID with RETURN_GENERATED_KEYS.
- Add timestamp (created_at) to track when feedback was submitted.
- Enforce uniqueness if buyers can review a product only once.

Scenario 30: Fetch overdue tasks for a user

In a team task manager, managers want to see tasks that a user hasn't completed on time. The tasks table contains due_date, and tasks overdue are those with due_date < today.

Answer:

I'll write a SELECT query filtering by owner ID and tasks where due_date is earlier than today. Using PreparedStatement, I'll bind both the user ID and current date securely.

Code:

```
import java.sql.*;
import java.time.LocalDate;

public class FetchOverdueTasks {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/appdb";
    private static final String DB_USER = "appuser";
    private static final String DB_PASS = "apppass";

    public static void main(String[] args) {
        fetchOverdueTasks(101); // Example owner ID
    }

    public static void fetchOverdueTasks(int ownerId) {
        String sql = "SELECT id, description, due_date " +
                     "FROM tasks " +
                     "WHERE owner_id = ? AND due_date < ? " +
                     "ORDER BY due_date";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, ownerId);
            ps.setDate(2, Date.valueOf(LocalDate.now()));

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    int id = rs.getInt("id");
                    String desc = rs.getString("description");
                    Date due = rs.getDate("due_date");

                    System.out.println("Task ID: " + id +
                                       ", Due: " + due +
                                       ", Desc: " + desc);
                }
            }
```

```
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- Filtering by due_date < today ensures only overdue tasks are retrieved.
- PreparedStatement allows safe reuse of the query and prevents injection.
- Sorting by due_date gives managers a prioritized view of the most overdue tasks first.
- try-with-resources handles resource cleanup automatically.

Future scope:

- Add a status column (e.g., completed) to filter out finished tasks.
- Allow filtering by project or team.
- Expose results via an API for dashboards.

Scenario 31: Migrate staff data from source DB to target DB (batched + transactional)

You're moving rows from hr_source.staff_src to hr_target.staff_tgt. This needs to be fast (batching) and safe (commit in chunks; rollback on failure).

Answer:

I'll stream rows from the source, insert into the target with a PreparedStatement, and execute in batches (e.g., 500). Auto-commit stays off on the target; after each batch I commit. If anything fails mid-way, I roll back the current batch so we never land in a half-migrated state.

Code:

```
import java.sql.*;

public class StaffDataMigration {
    private static final String SRC_URL = "jdbc:mysql://localhost:3306/hr_source";
    private static final String TGT_URL = "jdbc:mysql://localhost:3306/hr_target";
    private static final String DB_USER = "migrator";
    private static final String DB_PASS = "migratorpass";
    private static final int BATCH_SIZE = 500;

    public static void main(String[] args) {
        migrate();
    }

    public static void migrate() {
        String selectSql = "SELECT staff_id, full_name, role FROM staff_src";
        String insertSql = "INSERT INTO staff_tgt (staff_id, full_name, role) VALUES
(?, ?, ?)";

        int count = 0;
        try (Connection srcConn = DriverManager.getConnection(SRC_URL,
DB_USER, DB_PASS);
             Connection tgtConn = DriverManager.getConnection(TGT_URL,
DB_USER, DB_PASS);
             PreparedStatement tgtPs = tgtConn.prepareStatement(insertSql);
             Statement srcStmt = srcConn.createStatement(
                     ResultSet.TYPE_FORWARD_ONLY,
ResultSet.CONCUR_READ_ONLY)) {

            // help MySQL stream large result sets
            srcStmt.setFetchSize(Integer.MIN_VALUE);
            tgtConn.setAutoCommit(false);

            try (ResultSet rs = srcStmt.executeQuery(selectSql)) {
                while (rs.next()) {
                    tgtPs.setInt(1, rs.getInt("staff_id"));
```

```
                tgtPs.setString(2, rs.getString("full_name"));
                tgtPs.setString(3, rs.getString("role"));
                tgtPs.addBatch();
                count++;

                if (count % BATCH_SIZE == 0) {
                    tgtPs.executeBatch();
                    tgtConn.commit();
                }
            }

            if (count % BATCH_SIZE != 0) {
                tgtPs.executeBatch();
                tgtConn.commit();
            }
        } catch (SQLException batchErr) {
            tgtConn.rollback();
            throw batchErr;
        }

        System.out.println("Migration finished. Rows migrated: " + count);

    } catch (SQLException e) {
        e.printStackTrace();
    }
  }
}
```

Reasoning to interviewer:

- Batch inserts cut round-trips dramatically; committing per batch balances safety and speed.
- Disabling auto-commit on the target lets me atomically apply each batch and roll back on any error.
- Streaming the source (setFetchSize(Integer.MIN_VALUE) for MySQL) avoids loading everything into memory.
- Separate connections keep concerns clear; we never mix reads/writes on one connection.

Future scope:

- Add idempotency with INSERT ... ON DUPLICATE KEY UPDATE for re-runs.

• Capture rejects (bad rows) to a side table with the error message.
• Parallelize by key-range sharding if data volume is very large.

Scenario 32: Fetch a user's invoices from the previous calendar month

Finance needs a precise "previous month" report (e.g., all invoices from 01–31 Aug), not "last 30 days."

Answer:

I'll compute the exact start and end boundaries using YearMonth.now().minusMonths(1), then bind those dates into a parameterized query filtering issued_date BETWEEN ? AND ? for the given user_id. I'll order by issued_date so the report reads chronologically.

Code:

```
import java.sql.*;
import java.time.LocalDate;
import java.time.YearMonth;
import java.util.ArrayList;
import java.util.List;

class Invoice {
    final int id;
    final double amount;
    final Date issued;

    Invoice(int id, double amount, Date issued) {
        this.id = id;
        this.amount = amount;
        this.issued = issued;
    }

    @Override public String toString() {
        return "Invoice: " + id + ", Date: " + issued + ", Amount: " + amount;
    }
}

public class FetchLastMonthInvoices {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/billingdb";
    private static final String DB_USER = "billing";
    private static final String DB_PASS = "billingpass";

    public static void main(String[] args) {
        List<Invoice> invoices = fetchForPreviousMonth(101);
        invoices.forEach(System.out::println);
    }

    public static List<Invoice> fetchForPreviousMonth(int userId) {
        YearMonth lastMonth = YearMonth.now().minusMonths(1);
        LocalDate start = lastMonth.atDay(1);
```

```
        LocalDate end = lastMonth.atEndOfMonth();

        String sql = """
            SELECT invoice_id, amount, issued_date
            FROM invoices
            WHERE user_id = ? AND issued_date BETWEEN ? AND ?
            ORDER BY issued_date
        """;

        List<Invoice> out = new ArrayList<>();

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, userId);
            ps.setDate(2, Date.valueOf(start));
            ps.setDate(3, Date.valueOf(end));

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    out.add(new Invoice(
                            rs.getInt("invoice_id"),
                            rs.getDouble("amount"),
                            rs.getDate("issued_date")
                    ));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return out;
    }
}
```

Reasoning to interviewer:

- YearMonth gives exact calendar boundaries—no off-by-one mistakes vs. "last 30 days."
- PreparedStatement safely binds user_id and date range; the DB does the filtering.
- Ordering by date produces a clean report view and predictable pagination.
- Mapping to an Invoice type keeps the result usable across APIs/exports.

Future scope:

- Add a SUM(amount) footer for the month’s total.
- Allow arbitrary month selection (e.g., a dropdown of YearMonth).
- Expose CSV/Excel export for finance workflows.

Scenario 33: Update a user's profile information

In a user portal, people should be able to securely update their email and phone. The users table stores this info, and the update must be parameterized to prevent SQL injection.

Answer:

I'll prepare an UPDATE query with placeholders for email, phone, and user id. Using PreparedStatement ensures input is bound securely, and trimming values avoids storing accidental whitespace.

Code:

```
import java.sql.*;

public class UpdateUserProfile {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/appdb";
    private static final String DB_USER = "appuser";
    private static final String DB_PASS = "apppass";

    public static void main(String[] args) {
        updateProfile(101, "new.email@example.com", "9876543210");
    }

    public static void updateProfile(int userId, String newEmail, String newPhone) {
        String sql = "UPDATE users SET email = ?, phone = ? WHERE id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, newEmail.trim());
            ps.setString(2, newPhone.trim());
            ps.setInt(3, userId);

            int updated = ps.executeUpdate();
            if (updated > 0) {
                System.out.println("Profile updated successfully.");
            } else {
                System.out.println("No user found with id " + userId);
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- PreparedStatement avoids SQL injection and safely binds input values.
- executeUpdate() return value tells me if any rows were actually changed.
- Trimming email and phone prevents storing invalid whitespace.
- try-with-resources cleans up all database resources automatically.

Future scope:

- Add validation (email regex, phone format) before saving.
- Update an updated_at timestamp to track profile changes.
- Log updates in an audit table for security and compliance.

Scenario 34: Fetch all unread notifications for a user

In an alert system, users need to see their unread notifications first. The notifications table has is_read as a flag for this purpose.

Answer:

I'll write a SELECT query with filters for user_id and is_read = false. Ordering by created_at DESC ensures the newest notifications show up first.

Code:

```
import java.sql.*;

public class FetchUnreadNotifications {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/appdb";
    private static final String DB_USER = "appuser";
    private static final String DB_PASS = "apppass";

    public static void main(String[] args) {
        fetchUnread(101); // Example user ID
    }

    public static void fetchUnread(int userId) {
        String sql = "SELECT id, content, created_at " +
                     "FROM notifications " +
                     "WHERE user_id = ? AND is_read = ? " +
                     "ORDER BY created_at DESC";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, userId);
            ps.setBoolean(2, false);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    int id = rs.getInt("id");
                    String content = rs.getString("content");
                    Timestamp ts = rs.getTimestamp("created_at");

                    System.out.println("[" + ts + "] (" + id + ") " + content);
                }
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
```

```
}
```

Reasoning to interviewer:

- Filtering on is_read = false fetches only unread notifications.
- PreparedStatement keeps the query safe and reusable.
- Ordering by created_at DESC ensures the latest alerts appear at the top.
- try-with-resources ensures proper cleanup of the connection, statement, and result set.

Future scope:

- Add pagination so users can scroll through large notification sets.
- Include a LIMIT for mobile clients to reduce payload size.
- Allow marking notifications as read in the same transaction.

Scenario 35: Calculate average product rating

In an e-commerce system, customers rate products they purchase. To display product quality, the system needs to compute the average rating from the product_ratings table.

Answer:

I'll use the SQL aggregation function AVG(rating) with a parameterized query filtering by product_id. PreparedStatement ensures safe parameter binding, and the ResultSet will return a single row with the computed average.

Code:

```
import java.sql.*;

public class CalculateAverageRatingExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        double avg = calculateAverageRating(101); // Example product ID
        System.out.println("Average rating: " + avg);
    }

    public static double calculateAverageRating(int productId) {
        String sql = "SELECT AVG(rating) AS average_rating FROM product_ratings
WHERE product_id = ?";
        double average = 0.0;

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, productId);

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    average = rs.getDouble("average_rating");
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return average;
    }
}
```

Reasoning to interviewer:

• SQL’s AVG() function is efficient since aggregation is handled by the database.

• PreparedStatement ensures safe dynamic binding of product_id.

• Returning 0.0 when no reviews exist avoids null pointer issues in the app layer.

• This approach scales well even with large datasets since the DB engine optimizes aggregation.

Future scope:

• Round results (e.g., 2 decimal places) for UI display.

• Add filtering (e.g., average of ratings in the last 6 months).

• Cache averages in a summary table for high-traffic products.

Scenario 36: Retrieve active subscribers for a plan

In a subscription system, admins often need to see which users are actively subscribed to a given plan. A subscription is considered active if the current date is before or equal to its end_date.

Answer:

I'll prepare a query filtering by plan_id and end_date >= today. A PreparedStatement binds both values safely, and looping through the ResultSet prints each active subscriber's ID and start date.

Code:

```
import java.sql.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

class Subscriber {
    final int userId;
    final Date startDate;

    Subscriber(int userId, Date startDate) {
        this.userId = userId;
        this.startDate = startDate;
    }

    @Override
    public String toString() {
        return "User ID: " + userId + ", Start Date: " + startDate;
    }
}

public class RetrieveActiveSubscribersExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        List<Subscriber> list = getActiveSubscribers(101); // Example plan ID
        list.forEach(System.out::println);
    }

    public static List<Subscriber> getActiveSubscribers(int planId) {
        String sql = "SELECT user_id, start_date FROM subscriptions WHERE
plan_id = ? AND end_date >= ?";
        List<Subscriber> subscribers = new ArrayList<>();
```

```
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, planId);
            ps.setDate(2, Date.valueOf(LocalDate.now()));

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    subscribers.add(new Subscriber(
                            rs.getInt("user_id"),
                            rs.getDate("start_date")
                    ));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return subscribers;
    }
}
```

Reasoning to interviewer:

- Filtering by end_date >= currentDate ensures only currently active subscriptions are returned.
- Wrapping results in a Subscriber object makes the code reusable beyond just console output.
- PreparedStatement provides safety and flexibility when passing plan IDs dynamically.
- Using a list of subscribers supports reporting and downstream analytics.

Future scope:

- Add start_date <= currentDate to ensure future subscriptions don’t appear prematurely.
- Include plan details (plan name, price) in the query for richer reporting.
- Add pagination for very large subscriber lists.

Scenario 37: Retrieve all out-of-stock products

In an inventory management system, it's crucial to track which products are unavailable so procurement can reorder them. The products table stores stock quantities, and items with stock_quantity = 0 are considered out of stock.

Answer:

I'll write a SELECT query with a condition stock_quantity = 0. Since no dynamic parameters are needed, PreparedStatement is still used for consistency and security. Iterating through the ResultSet prints product details.

Code:

```
import java.sql.*;

public class RetrieveOutOfStockProductsExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        fetchOutOfStockProducts();
    }

    public static void fetchOutOfStockProducts() {
        String sql = "SELECT product_id, name, price FROM products WHERE
stock_quantity = 0";

        try (Connection connection = DriverManager.getConnection(DB_URL,
DB_USER, DB_PASS);
             PreparedStatement ps = connection.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                int productId = rs.getInt("product_id");
                String name = rs.getString("name");
                double price = rs.getDouble("price");

                System.out.println("Product ID: " + productId +
                                   ", Name: " + name +
                                   ", Price: " + price);
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- The fixed filter stock_quantity = 0 directly returns only unavailable products.
- PreparedStatement is used for safety and uniformity, even when no parameters are required.
- Using try-with-resources ensures connections, statements, and result sets are closed properly.
- This approach is efficient for generating out-of-stock reports for procurement teams.

Future scope:

- Extend the query to include products with stock_quantity < threshold (low stock alerts).
- Add sorting by product name or category for easier reporting.
- Integrate results into a dashboard or automated reorder workflow.

Scenario 38: Retrieve tasks for a user ordered by priority

In a task management system, tasks need to be shown in order of importance so users know what to tackle first. The tasks table includes task details along with a numeric priority.

Answer:

I'll prepare a SELECT query filtering by user_id and ordering by priority DESC. This ensures higher-priority tasks appear at the top. A PreparedStatement safely binds the user ID, and results are looped through and displayed.

Code:

```
import java.sql.*;

public class RetrieveTasksByPriorityExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        fetchTasksByPriority(101); // Example user ID
    }

    public static void fetchTasksByPriority(int userId) {
        String sql = "SELECT task_id, task_description, priority " +
                     "FROM tasks WHERE user_id = ? ORDER BY priority DESC";

        try (Connection connection = DriverManager.getConnection(DB_URL,
DB_USER, DB_PASS);
             PreparedStatement ps = connection.prepareStatement(sql)) {

            ps.setInt(1, userId);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    int taskId = rs.getInt("task_id");
                    String desc = rs.getString("task_description");
                    int priority = rs.getInt("priority");

                    System.out.println("Task ID: " + taskId +
                                       ", Description: " + desc +
                                       ", Priority: " + priority);
                }
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
```

```
    }
}
```

Reasoning to interviewer:

- Filtering by user_id ensures only that user's tasks are retrieved.
- ORDER BY priority DESC makes higher-priority tasks show up first, which aligns with real-world usage.
- PreparedStatement avoids SQL injection and supports reusability across different users.
- try-with-resources ensures all DB resources are properly closed.

Future scope:

- Add filters (e.g., show only incomplete tasks).
- Support pagination for users with many tasks.
- Allow sorting by additional fields like due date or status.

Scenario 39: Retrieve movies of a specific genre released in the past year

For a movie streaming platform's recommendation system, users should see only relevant and recent content. The movies table includes genre and release_date, which can be used to filter movies by both category and recency.

Answer:
I'll write a parameterized query filtering by genre and release_date >= one year ago. I'll use LocalDate.now().minusYears(1) in Java to dynamically calculate the date.

Code:

```
import java.sql.*;
import java.time.LocalDate;

public class RetrieveMoviesByGenreExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        fetchMovies("Action"); // Example genre
    }

    public static void fetchMovies(String genre) {
        String sql = "SELECT movie_id, title, genre, release_date " +
                     "FROM movies WHERE genre = ? AND release_date >= ?";

        try (Connection connection = DriverManager.getConnection(DB_URL,
DB_USER, DB_PASS);
             PreparedStatement ps = connection.prepareStatement(sql)) {

            ps.setString(1, genre);
            ps.setDate(2, Date.valueOf(LocalDate.now().minusYears(1)));

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    int movieId = rs.getInt("movie_id");
                    String title = rs.getString("title");
                    String g = rs.getString("genre");
                    Date releaseDate = rs.getDate("release_date");

                    System.out.println("Movie ID: " + movieId +
                                       ", Title: " + title +
                                       ", Genre: " + g +
                                       ", Release Date: " + releaseDate);
                }
            }
```

```
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- Using LocalDate.now().minusYears(1) ensures the time window always updates dynamically.
- PreparedStatement allows safe binding of both genre and date.
- Iterating over the ResultSet prints all matching movies, which can then be plugged into recommendation logic.
- This approach can scale by adding more filters like language, rating, or region.

Future scope:

- Add sorting by popularity or release date.
- Limit results with LIMIT ? for performance in large datasets.
- Pre-compute recommendations in a cache for faster responses.

Scenario 40: Update loyalty points for a customer after a purchase

In a loyalty program, each purchase should immediately update the user's accumulated points. The customer_rewards table stores customer ID and their total points.

Answer:

I'll use a parameterized UPDATE query with points = points + ? to increment the points. After execution, I'll check rowsUpdated to confirm the customer exists.

Code:

```
import java.sql.*;

public class UpdateLoyaltyPointsExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        addPoints(101, 100); // Example: add 100 points to customer 101
    }

    public static void addPoints(int customerId, int earnedPoints) {
        String sql = "UPDATE customer_rewards SET points = points + ? WHERE
customer_id = ?";

        try (Connection connection = DriverManager.getConnection(DB_URL,
DB_USER, DB_PASS);
             PreparedStatement ps = connection.prepareStatement(sql)) {

            ps.setInt(1, earnedPoints);
            ps.setInt(2, customerId);

            int rowsUpdated = ps.executeUpdate();
            if (rowsUpdated > 0) {
                System.out.println("Loyalty points updated successfully.");
            } else {
                System.out.println("Customer not found.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- Using points = points + ? is atomic—so concurrent purchases won't overwrite each other's updates.
- PreparedStatement ensures secure binding of inputs.
- Checking rowsUpdated prevents silently failing when the customer doesn't exist.
- This method integrates easily into the purchase workflow.

Future scope:

- Wrap updates in a transaction along with purchase insertion to ensure consistency.
- Log each points update in a rewards_history table for auditing.
- Add business rules like bonus multipliers for premium users.

Scenario 41: Insert an employee and log the action atomically

In an employee management system, when a new employee is added, the action must also be logged in an audit_log table. Both inserts should succeed together or not happen at all.

Answer:

I'll disable auto-commit, perform both inserts within a single transaction, and then explicitly commit. If any insert fails, I'll roll back to ensure consistency.

Code:

```
import java.sql.*;

public class TransactionExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS)) {
            conn.setAutoCommit(false); // Start transaction

            try (PreparedStatement empStmt = conn.prepareStatement(
                    "INSERT INTO employees (name, position) VALUES (?, ?)");
                 PreparedStatement auditStmt = conn.prepareStatement(
                    "INSERT INTO audit_log (action, timestamp) VALUES (?, NOW())")) {

                empStmt.setString(1, "John Doe");
                empStmt.setString(2, "Developer");
                empStmt.executeUpdate();

                auditStmt.setString(1, "Inserted new employee John Doe");
                auditStmt.executeUpdate();

                conn.commit(); // Commit if both succeed
                System.out.println("Transaction committed successfully.");

            } catch (SQLException ex) {
                conn.rollback(); // Undo changes if any step fails
                System.out.println("Transaction rolled back due to error.");
                ex.printStackTrace();
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
```

```
}
```

Reasoning to interviewer:

- Setting autoCommit = false ensures operations are grouped into one transaction.
- Using commit/rollback ensures atomicity—either both inserts succeed or both fail.
- Wrapping inserts in try-with-resources avoids leaks and keeps the code clean.
- This is essential in real-world systems where logs and data must stay in sync.

Future scope:

- Extend logging to include user performing the action, not just the description.
- Add error codes/messages to the audit log for better debugging.
- Use batch inserts if multiple employees are added at once.

Scenario 42: Call a stored procedure using CallableStatement

Sometimes, database logic is encapsulated in stored procedures. For example, GetCustomerOrders returns all orders for a given customer ID. The Java code should call it using JDBC.

Answer:

I'll use CallableStatement, which is designed for calling stored procedures. I'll bind the customer ID parameter, execute the procedure, and then process the ResultSet like a normal query.

Code:

```
import java.sql.*;

public class CallableStatementExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        callGetCustomerOrders(101); // Example customer ID
    }

    public static void callGetCustomerOrders(int customerId) {
        String sql = "{CALL GetCustomerOrders(?)}";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             CallableStatement cs = conn.prepareCall(sql)) {

            cs.setInt(1, customerId);

            try (ResultSet rs = cs.executeQuery()) {
                while (rs.next()) {
                    int orderId = rs.getInt("order_id");
                    Date orderDate = rs.getDate("order_date");
                    System.out.println("Order ID: " + orderId + ", Date: " + orderDate);
                }
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- CallableStatement is the standard JDBC interface for calling stored procedures.
- The syntax {CALL procedureName(?...)} is used to invoke stored procs with parameters.
- Parameters can be both input (e.g., customer ID) and output (e.g., total order count).
- ResultSet processing works the same way as a normal SELECT query.

Future scope:

- Handle OUT or INOUT parameters using registerOutParameter.
- Wrap the procedure call inside a service layer for reusability.
- Use stored procedures for performance-heavy queries to reduce application-side logic.

Scenario 43: Batch update multiple students' GPA efficiently

When you need to update many rows (like GPAs for a cohort), sending one SQL call per row kills performance. Batching lets the driver send a packet of parameter sets in one go and commit them together.

Answer:

I'll use a single prepared UPDATE with placeholders, turn off auto-commit, add many parameter sets via addBatch(), then call executeBatch() once and commit. If anything fails, I'll roll back so the batch is all-or-nothing.

Code:

```
import java.sql.*;
import java.util.Arrays;

public class BatchUpdateExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        String sql = "UPDATE students SET gpa = ? WHERE student_id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            conn.setAutoCommit(false); // do the whole batch atomically

            // 101 -> 3.5
            ps.setDouble(1, 3.5);
            ps.setInt(2, 101);
            ps.addBatch();

            // 102 -> 3.8
            ps.setDouble(1, 3.8);
            ps.setInt(2, 102);
            ps.addBatch();

            // 103 -> 4.0
            ps.setDouble(1, 4.0);
            ps.setInt(2, 103);
            ps.addBatch();

            int[] counts = ps.executeBatch();
            conn.commit();
```

```
            System.out.println("Batch update completed. Rows affected (per stmt): " +
Arrays.toString(counts));
        } catch (SQLException e) {
            e.printStackTrace(); // in real code: log + attempt rollback
        }
    }
}
```

Reasoning to interviewer:

- One prepared statement + many parameter sets minimizes round-trips and plan compilations.
- Disabling auto-commit groups the updates; if one fails, we can roll back the whole batch.
- executeBatch() returns per-item counts so we can verify effects and detect failures.
- This pattern scales to thousands of rows; MySQL can further speed this up with rewriteBatchedStatements=true.

Future scope:

- Chunk very large updates (e.g., 1k–5k rows per batch) to control memory/lock pressure.
- Capture failures with BatchUpdateException and retry only failed items.
- Move the GPA list into a CSV import or staging table + single UPDATE ... JOIN for even faster bulk ops.

Scenario 44: Use a JDBC connection pool (DataSource) instead of new connections

Opening a new DB connection per request is expensive. A pool keeps a small set of warm connections and hands them out on demand.

Answer:

I'll configure a HikariDataSource (fast, production-grade), then grab connections via dataSource.getConnection(). The pool manages lifecycles; my code just borrows and returns connections via try-with-resources.

Code:

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.*;

public class ConnectionPoolingExample {
    // Typically a singleton in your app
    private static HikariDataSource ds;

    static {
        HikariConfig cfg = new HikariConfig();
        cfg.setJdbcUrl("jdbc:mysql://localhost:3306/mydb");
        cfg.setUsername("user");
        cfg.setPassword("password");
        cfg.setMaximumPoolSize(10);
        cfg.setMinimumIdle(2);
        cfg.setIdleTimeout(300_000);   // 5 min
        cfg.setConnectionTimeout(30_000);
        cfg.setPoolName("mydb-pool");

        // Optional MySQL perf tweaks:
        // cfg.addDataSourceProperty("cachePrepStmts", "true");
        // cfg.addDataSourceProperty("prepStmtCacheSize", "250");
        // cfg.addDataSourceProperty("prepStmtCacheSqlLimit", "2048");

        ds = new HikariDataSource(cfg);
    }

    public static void main(String[] args) {
        // Example usage: simple SELECT through the pool
        String sql = "SELECT employee_id, name FROM employees ORDER BY
name";
        try (Connection conn = ds.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
```

```
            System.out.println(rs.getInt("employee_id") + " - " +
rs.getString("name"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // In a real app, close the pool at application shutdown (not per request):
            if (ds != null) ds.close();
        }
    }
}
```

Reasoning to interviewer:

- DataSource abstracts connection creation and pooling; acquiring a connection is O(milliseconds) vs. expensive TCP handshakes.
- HikariCP handles validation, leak detection, and back-pressure; my code just uses try-with-resources to return connections to the pool.
- Prepared statements are cached by the pool/driver, reducing parse/plan overhead.

Future scope:

- Externalize pool settings (env vars, config file) for different environments.
- Add health checks and metrics (Micrometer) to watch pool utilization and wait times.
- Use separate pools for read/write traffic or different schemas to isolate workloads.

Scenario 45: Scrollable and updatable ResultSet for library records

In a desktop-based library management app, the librarian wants to navigate book records forward and backward without rerunning queries, and also update book titles directly from the result set.

Answer:

I'll create a Statement with TYPE_SCROLL_SENSITIVE (for bi-directional navigation and reflecting DB changes) and CONCUR_UPDATABLE (for in-place updates). This allows moving the cursor backward and forward and calling updateRow() to persist changes.

Code:

```
import java.sql.*;

public class ScrollableUpdatableExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/librarydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             Statement stmt = conn.createStatement(
                     ResultSet.TYPE_SCROLL_SENSITIVE,
                     ResultSet.CONCUR_UPDATABLE)) {

            ResultSet rs = stmt.executeQuery("SELECT * FROM books");

            // Move to the end and iterate backwards
            rs.afterLast();
            while (rs.previous()) {
                String title = rs.getString("title");
                System.out.println("Book: " + title);

                if ("Old Title".equals(title)) {
                    rs.updateString("title", "Updated Title");
                    rs.updateRow(); // Persist the update
                }
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• TYPE_SCROLL_SENSITIVE allows cursor navigation in both directions and reflects DB changes in the result set.

• CONCUR_UPDATABLE lets me modify rows directly in the result set and sync them back to the database.

• This design is practical for interactive UIs (like librarian consoles) where browsing and editing records happen in the same view.

• Calling updateRow() immediately persists the modified data.

Future scope:

• Add support for inserting and deleting rows directly through the ResultSet.

• Use pagination or fetch size tuning for very large tables.

• Combine with GUI components (like JTable in Swing) to create editable grids.

Scenario 46: Prevent SQL injection in flight search

In a flight booking system, users enter departure and destination cities to search flights. The system must prevent SQL injection attacks when executing the query.

Answer:

I'll use a PreparedStatement with placeholders instead of string concatenation. User inputs are bound via setString(), so they are treated as values, not executable SQL.

Code:

```
import java.sql.*;

public class SqlInjectionPreventionExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/traveldb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        searchFlights("New York", "London");
    }

    public static void searchFlights(String departure, String destination) {
        String sql = "SELECT flight_id, departure_city, destination_city " +
                     "FROM flights WHERE departure_city = ? AND destination_city = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, departure);
            ps.setString(2, destination);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    System.out.println("Flight ID: " + rs.getInt("flight_id") +
                                       ", Departure: " + rs.getString("departure_city") +
                                       ", Destination: " + rs.getString("destination_city"));
                }
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- PreparedStatement ensures user input is safely bound and cannot alter the SQL structure.
- This eliminates the risk of injection like "' OR '1'='1" that could bypass filters.
- It’s a best practice whenever user input is used in queries (search, login, etc.).
- The approach is portable across all JDBC-supported databases.

Future scope:

- Validate inputs (e.g., check city codes against a whitelist).
- Implement parameterized stored procedures for complex queries.
- Add query caching for frequent flight searches to reduce DB load.

Scenario 47: Using Savepoint for partial rollback in payment transactions

In an online payment system, we may deduct money from the buyer but fail when crediting the seller. Instead of rolling back the entire transaction, we want to roll back only part of it, giving us finer control.

Answer:

I'll disable auto-commit, perform the buyer deduction, then set a Savepoint. If the seller credit fails, I'll roll back only to that savepoint so the deduction remains, but no incorrect credit occurs.

Code:

```
import java.sql.*;

public class SavepointExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/paymentsdb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             Statement stmt = conn.createStatement()) {

            conn.setAutoCommit(false);

            // Step 1: Deduct from buyer
            stmt.executeUpdate("UPDATE accounts SET balance = balance - 500
WHERE account_id = 101");

            // Savepoint after deduction
            Savepoint sp = conn.setSavepoint("AfterBuyerDeduction");

            try {
                // Step 2: Attempt to credit seller
                stmt.executeUpdate("UPDATE accounts SET balance = balance + 500
WHERE account_id = 9999");

                conn.commit();
                System.out.println("Transaction completed successfully.");

            } catch (SQLException ex) {
                // Roll back only to savepoint (keeps buyer deduction)
                conn.rollback(sp);
                conn.commit();
                System.out.println("Rolled back to savepoint. Buyer deduction kept,
seller credit skipped.");
```

```
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- Savepoint allows finer-grained rollback, not just full rollback.
- By setting the savepoint after the buyer deduction, we preserve that step even if the seller update fails.
- Committing after rollback ensures the valid part of the transaction (buyer deduction) is finalized.
- This pattern is common when parts of a workflow are optional or failure-tolerant.

Future scope:

- Implement compensating transactions (e.g., refund buyer automatically if seller credit fails).
- Use named savepoints for clarity in larger workflows with multiple steps.
- Log rollback events to an audit table for traceability.

Scenario 48: Calling a stored procedure with OUT parameter (applicant count)

In a recruitment system, a stored procedure getApplicantCount takes a job ID and returns the total number of applicants. We need to use JDBC to call it and retrieve the output.

Answer:

I'll use CallableStatement. The job ID is bound as an input parameter, and the applicant count is registered as an OUT parameter. After executing, I'll fetch the OUT parameter with getInt().

Code:

```
import java.sql.*;

public class CallableStatementOutputExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/hrdb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        callApplicantCount(202); // Example job ID
    }

    public static void callApplicantCount(int jobId) {
        String sql = "{ call getApplicantCount(?, ?) }";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             CallableStatement cs = conn.prepareCall(sql)) {

            cs.setInt(1, jobId);                // IN parameter
            cs.registerOutParameter(2, Types.INTEGER); // OUT parameter

            cs.execute();

            int count = cs.getInt(2);
            System.out.println("Total applicants for job ID " + jobId + ": " + count);

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• CallableStatement is the correct JDBC API for stored procedures.

• registerOutParameter tells JDBC the data type of the OUT parameter so it can retrieve it.

• After execution, OUT values are fetched via getXXX() methods.

• This improves performance by reusing DB-side logic and avoids duplicating logic in the Java app.

Future scope:

• Support multiple OUT parameters for richer results (e.g., applicant count + last applied date).

• Use INOUT parameters if the stored procedure modifies and returns input values.

• Integrate this into reporting dashboards for recruiters.

Scenario 49: Batch-insert shipment tracking updates from CSV

Nightly logistics jobs should insert thousands of tracking updates efficiently. Doing one INSERT per row is slow; batching cuts round-trips and speeds things up.

Answer:

I'll parse the CSV, use a single PreparedStatement, add parameter sets with addBatch(), and execute in chunks inside an explicit transaction. If anything fails, I'll roll back the whole load so we don't end up half-imported.

Code:

```
import java.sql.*;
import java.util.*;
import java.nio.file.*;
import java.io.*;

public class BatchInsertShipments {
    private static final String DB_URL =
"jdbc:mysql://localhost:3306/logistics?rewriteBatchedStatements=true";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "pass123";
    private static final int BATCH_SIZE = 1000;

    public static void main(String[] args) throws Exception {
        // Example data; in production read from CSV file
        List<String[]> shipmentData = Arrays.asList(
            new String[]{"ORD1001", "Shipped"},
            new String[]{"ORD1002", "In Transit"},
            new String[]{"ORD1003", "Delivered"}
        );

        String sql = "INSERT INTO shipment_tracking (order_id, status) VALUES (?,
?)";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            conn.setAutoCommit(false);
            int count = 0;

            for (String[] rec : shipmentData) {
                ps.setString(1, rec[0]); // order_id
                ps.setString(2, rec[1]); // status
                ps.addBatch();

                if (++count % BATCH_SIZE == 0) {
                    ps.executeBatch();
```

```
                conn.commit(); // commit per chunk
            }
        }

        if (count % BATCH_SIZE != 0) {
            ps.executeBatch();
            conn.commit();
        }

        System.out.println("Inserted " + count + " tracking records.");
    } catch (SQLException e) {
        e.printStackTrace();
        // In a real job, log and alert; job scheduler can retry
    }
  }
}
```

Reasoning to interviewer:

- One prepared INSERT + many parameter sets minimizes network hops and statement parsing.
- rewriteBatchedStatements=true (MySQL) lets the driver send batches as a single multi-values INSERT.
- Chunking + commits per chunk balance performance with failure blast radius.
- Turning off auto-commit ensures we can roll back a partial chunk on error.

Future scope:

- Stream-read the CSV (e.g., Files.lines) to avoid loading it all in memory.
- Use a staging table + single MERGE/INSERT … ON DUPLICATE KEY UPDATE if you need upserts.
- Add basic validation and a “rejects” file for bad rows with reasons.

Scenario 50: All-or-nothing fund transfers (transactional batch)

Multiple transfers run together (debit A/credit B pairs). If any pair fails, none of the transfers should persist.

Answer:

I'll wrap the whole set in a single transaction (setAutoCommit(false)), execute each debit/credit pair using PreparedStatements, and only commit() at the end. On any failure, I'll rollback() the entire transaction.

Code:

```
import java.sql.*;
import java.util.List;

public class FundTransferBatch {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/bankdb";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "pass123";

    static class Transfer {
        final int fromId, toId;
        final double amount;
        Transfer(int fromId, int toId, double amount) { this.fromId = fromId; this.toId =
toId; this.amount = amount; }
    }

    public static void main(String[] args) {
        List<Transfer> batch = List.of(
            new Transfer(101, 202, 500),
            new Transfer(103, 204, 300)
        );
        boolean ok = processTransfers(batch);
        System.out.println(ok ? "All transfers completed successfully." : "Transaction
failed. All changes rolled back.");
    }

    public static boolean processTransfers(List<Transfer> transfers) {
        String debitSql  = "UPDATE accounts SET balance = balance - ? WHERE
account_id = ? AND balance >= ?";
        String creditSql = "UPDATE accounts SET balance = balance + ? WHERE
account_id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement debit  = conn.prepareStatement(debitSql);
             PreparedStatement credit = conn.prepareStatement(creditSql)) {

            conn.setAutoCommit(false);
```

```
            for (Transfer t : transfers) {
                // debit (with sufficient-balance guard)
                debit.setDouble(1, t.amount);
                debit.setInt(2, t.fromId);
                debit.setDouble(3, t.amount);
                int debited = debit.executeUpdate();
                if (debited == 0) throw new SQLException("Insufficient funds or from-
account not found: " + t.fromId);

                // credit
                credit.setDouble(1, t.amount);
                credit.setInt(2, t.toId);
                int credited = credit.executeUpdate();
                if (credited == 0) throw new SQLException("To-account not found: " +
t.toId);
            }

            conn.commit();
            return true;

        } catch (SQLException e) {
            e.printStackTrace();
            try {
                // Attempt rollback of everything
                DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS).rollback(); // not ideal; better: keep conn ref to rollback
            } catch (SQLException ignore) {}
            return false;
        }
    }
}
```

Reasoning to interviewer:

- A single transaction guarantees atomicity: either all (every debit/credit pair) or none.
- Using two prepared statements reuses compiled plans and keeps code clear.
- Guarding the debit with balance >= ? prevents overdrafts inside the same statement, avoiding racey read-then-write checks.
- Rolling back on the first failure keeps the ledger consistent.

Future scope:

• Add per-transfer savepoints if you want to continue remaining transfers even when one pair fails—but that changes the business rule.
• Introduce idempotency keys to prevent duplicate execution on retries.
• Consider serializable/SELECT … FOR UPDATE if you need stricter isolation for high contention accounts.

Scenario 51: Call a stored procedure to get employee details

In an HR system, employee details like name, department, and salary are encapsulated in a stored procedure (get_employee_details). The application should call it with an employee ID and display the result.

Answer:

I'll use CallableStatement to call the procedure. Since this procedure returns a ResultSet, I can handle it just like a normal query with executeQuery().

Code:

```
import java.sql.*;

public class CallStoredProcedureExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/hrdb";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "pass123";

    public static void main(String[] args) {
        getEmployeeDetails(101); // Example employee ID
    }

    public static void getEmployeeDetails(int empId) {
        String sql = "{ call get_employee_details(?) }";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             CallableStatement cs = conn.prepareCall(sql)) {

            cs.setInt(1, empId);

            try (ResultSet rs = cs.executeQuery()) {
                while (rs.next()) {
                    String name = rs.getString("name");
                    String dept = rs.getString("department");
                    double salary = rs.getDouble("salary");

                    System.out.println("Name: " + name +
                                       ", Department: " + dept +
                                       ", Salary: " + salary);
                }
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- CallableStatement is the standard way to invoke stored procedures in JDBC.
- Input parameters are bound via setXXX() methods, here setInt().
- Since the procedure returns a ResultSet, I can handle it like a SELECT query.
- This allows encapsulating business logic in the DB while keeping the Java code clean.

Future scope:

- Add OUT parameters (e.g., employee status or bonus eligibility) using registerOutParameter.
- Handle cases where no employee is found gracefully.
- Wrap this in a DAO/service layer for reuse across the HR app.

Scenario 52: Scrollable ResultSet for browsing books

In a bookstore reporting tool, admins want to see books sorted by release date but also move backward and forward without re-running queries.

Answer:

I'll create a Statement with TYPE_SCROLL_INSENSITIVE and CONCUR_READ_ONLY. This lets me scroll through results in both directions and navigate randomly.

Code:

```
import java.sql.*;

public class ScrollableResultSetExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/bookstore";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "pass123";

    public static void main(String[] args) {
        fetchBooks();
    }

    public static void fetchBooks() {
        String sql = "SELECT title, release_date FROM books ORDER BY
release_date DESC";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             Statement stmt = conn.createStatement(
                     ResultSet.TYPE_SCROLL_INSENSITIVE,
                     ResultSet.CONCUR_READ_ONLY);
             ResultSet rs = stmt.executeQuery(sql)) {

            // Start from the end and move backwards
            rs.afterLast();
            while (rs.previous()) {
                System.out.println(rs.getString("title") +
                                   " - " + rs.getDate("release_date"));
            }

            // Example: jump to 3rd record
            if (rs.absolute(3)) {
                System.out.println("3rd book: " + rs.getString("title"));
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
```

```
    }
}
```

Reasoning to interviewer:

- TYPE_SCROLL_INSENSITIVE allows navigation both forward and backward, but doesn't reflect DB changes after the ResultSet was created (better performance).
- CONCUR_READ_ONLY ensures data safety as we're only reading, not updating.
- Methods like previous(), absolute(), and relative() give flexible navigation.
- This avoids firing new queries when moving around the dataset.

Future scope:

- Use TYPE_SCROLL_SENSITIVE if real-time reflection of DB changes is needed.
- Add pagination (e.g., jump to record 50–100).
- Integrate with a GUI table (like Swing JTable) for interactive reporting.

Scenario 53: Preventing SQL injection in legacy query building

The old system was building SQL like "SELECT * FROM customers WHERE email = '" + userInput + "'", which is dangerous if someone types malicious input. The fix is to parameterize the query with PreparedStatement.

Answer:

Instead of string concatenation, I'll bind the email safely with a ? placeholder.

Code:

```
import java.sql.*;

public class PreventSQLInjectionExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/shopdb";
        String username = "root";
        String password = "pass123";

        String userInputEmail = "test@example.com"; // Simulated user input

        try (Connection conn = DriverManager.getConnection(url, username,
password)) {
            String sql = "SELECT customer_id, name, email FROM customers
WHERE email = ?";
            try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
                pstmt.setString(1, userInputEmail);

                try (ResultSet rs = pstmt.executeQuery()) {
                    if (rs.next()) {
                        System.out.println("Customer: " + rs.getString("name")
                                + " (" + rs.getString("email") + ")");
                    } else {
                        System.out.println("No customer found with email: " +
userInputEmail);
                    }
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- String concatenation treats user input as SQL code, enabling injection. With PreparedStatement, the SQL is precompiled and user input is sent as data,

not part of the query structure.

• This shields against attacks like: "' OR 1=1 --".

• Using parameter binding also improves performance since the database can cache the query plan for repeated execution.

• Returning selected columns instead of * is more efficient and clearer.

Future scope:

• Apply the same refactor to all queries that include user input (login, search, filtering).

• Add input validation (like checking email format) before sending to DB.

• For very sensitive contexts, consider stored procedures or ORM frameworks (like JPA/Hibernate) which handle parameterization internally.

Scenario 54: Bulk price updates from CSV (batched + transactional)

In an e-commerce back office, thousands of products get repriced nightly. Running one update per row is slow and noisy. Batching the updates and committing in chunks keeps it fast and safe.

Answer:

I'll use a single PreparedStatement (UPDATE products SET price=? WHERE product_id=?), turn off auto-commit, add parameter sets via addBatch(), execute every N rows (e.g., 1000), and commit() per chunk. If a batch fails, I'll rollback() so we don't leave partial updates.

Code:

```
import java.sql.*;
import java.util.List;

public class BatchUpdateExample {
    private static final String DB_URL =
"jdbc:mysql://localhost:3306/mydb?rewriteBatchedStatements=true";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";
    private static final int BATCH_SIZE = 1000;

    // Imagine this comes from a CSV parser: (productId, newPrice)
    record PriceUpdate(int productId, double price) {}
    public static void main(String[] args) {
        List<PriceUpdate> updates = List.of(
            new PriceUpdate(1, 149.99),
            new PriceUpdate(2, 249.99)
            // ... thousands more
        );

        String sql = "UPDATE products SET price = ? WHERE product_id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            conn.setAutoCommit(false);

            int count = 0;
            for (PriceUpdate u : updates) {
                ps.setDouble(1, u.price());
                ps.setInt(2, u.productId());
                ps.addBatch();

                if (++count % BATCH_SIZE == 0) {
                    ps.executeBatch();
```

```
                conn.commit();              // commit this chunk
            }
        }

        if (count % BATCH_SIZE != 0) {
            ps.executeBatch();              // flush any remainder
            conn.commit();
        }

        System.out.println("Batch update completed successfully for " + count + "
rows.");
    } catch (SQLException e) {
        e.printStackTrace();                // in real code: log + alert
        // If you keep a ref to the same Connection, call conn.rollback() here
    }
  }
}
```

Reasoning to interviewer:

- One prepared statement + many parameter sets slashes round-trips and parse time.
- rewriteBatchedStatements=true lets MySQL collapse batches for extra speed.
- Chunked commits keep transactions manageable and reduce lock contention.
- Disabling auto-commit lets me roll back the current chunk if anything fails.

Future scope:

- Stream CSV rows to avoid loading everything in memory; validate each row (numeric price, existing product).
- Use a staging table + UPDATE ... JOIN for the fastest single-shot bulk update.
- Emit a “rejects” file for rows that failed validation or foreign-key checks.

Scenario 55: Scrollable + updatable ResultSet for interactive editing

A report viewer needs to scroll forward/backward through results and occasionally edit rows inline—without firing separate UPDATE statements.

Answer:

I'll request a scrollable, updatable ResultSet from the Statement, select from a single base table including the primary key, and then use navigation methods (first(), last(), absolute()) plus updateXxx(...) and updateRow() to persist inline edits.

Code:

```
import java.sql.*;

public class ScrollableResultSetExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        String sql = "SELECT product_id, price FROM products ORDER BY
product_id"; // include PK

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             Statement stmt = conn.createStatement(
                     ResultSet.TYPE_SCROLL_INSENSITIVE,   // navigate both ways;
stable snapshot
                     ResultSet.CONCUR_UPDATABLE);         // allow in-place edits
             ResultSet rs = stmt.executeQuery(sql)) {

            // navigate
            if (rs.last()) {
                System.out.println("Last row Product ID: " + rs.getInt("product_id"));
            }
            if (rs.first()) {
                System.out.println("First row Product ID: " + rs.getInt("product_id"));
            }

            // jump to row 3 and edit
            if (rs.absolute(3)) {
                rs.updateDouble("price", 199.99);
                rs.updateRow(); // persist change for current row
                System.out.println("Updated price of row 3.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
```

```
    }
}
```

Reasoning to interviewer:

- TYPE_SCROLL_INSENSITIVE enables forward/backward/random access without re-querying; data is a consistent snapshot.
- CONCUR_UPDATABLE lets me call updateXxx + updateRow() to persist changes directly from the cursor.
- Selecting from a single table and including the primary key is key for updatable result sets in most drivers.
- This pattern fits desktop/embedded admin tools where browse + inline-edit flows are common.

Future scope:

- For live reflections of concurrent DB changes, switch to TYPE_SCROLL_SENSITIVE (with performance trade-offs).
- Add insertRow()/deleteRow() to support full CRUD.
- For large datasets, prefer server-side pagination (LIMIT/OFFSET) plus dedicated UPDATE statements to reduce memory usage.

Scenario 56: Generate schema documentation using DatabaseMetaData and ResultSetMetaData

When building a schema documentation tool, you need to explore database tables and their columns without hardcoding names. JDBC provides metadata APIs to make this possible.

Answer:

I'll use DatabaseMetaData to get all table names and then ResultSetMetaData to extract column details (names and types) for each table dynamically.

Code:

```
import java.sql.*;

public class MetadataExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS)) {

            DatabaseMetaData dbMeta = conn.getMetaData();

            // Fetch all tables
            ResultSet tables = dbMeta.getTables(null, null, "%", new
String[]{"TABLE"});
            while (tables.next()) {
                String tableName = tables.getString("TABLE_NAME");
                System.out.println("Table: " + tableName);

                // Fetch one row just to get column metadata
                try (Statement stmt = conn.createStatement();
                     ResultSet rs = stmt.executeQuery("SELECT * FROM " + tableName
+ " LIMIT 1")) {

                    ResultSetMetaData rsMeta = rs.getMetaData();
                    for (int i = 1; i <= rsMeta.getColumnCount(); i++) {
                        System.out.println("   Column: " + rsMeta.getColumnName(i)
                                + " (" + rsMeta.getColumnTypeName(i) + ")");
                    }
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
```

```
}
```

Reasoning to interviewer:

• DatabaseMetaData is database-level metadata: table names, indexes, keys, etc.

• ResultSetMetaData is result-level metadata: column names, types, precision.

• By querying SELECT * ... LIMIT 1, I avoid fetching all rows—only metadata.

• This design auto-adapts to schema changes and is ideal for tools like schema explorers, migration utilities, or admin dashboards.

Future scope:

• Extend to include primary keys, foreign keys, and constraints using DatabaseMetaData.getPrimaryKeys() and getImportedKeys().

• Export metadata to JSON, HTML, or Markdown for human-readable schema documentation.

• Integrate into CI/CD pipelines to track schema drift across environments.

Scenario 57: Call stored procedure with IN and OUT parameters

In a travel booking app, discount rules are in a stored procedure (calculate_discount). It takes passenger type and booking date as input and returns the discounted ticket price as output.

Answer:

I'll call this stored procedure with CallableStatement. I'll set input parameters using setXXX(), register the output parameter, then retrieve it after execution.

Code:

```
import java.sql.*;

public class CallableStatementExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        String sql = "{ call calculate_discount(?, ?, ?) }";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             CallableStatement cs = conn.prepareCall(sql)) {

            cs.setString(1, "ADULT");                          // IN param: passenger type
            cs.setDate(2, Date.valueOf("2025-08-15"));         // IN param: booking date
            cs.registerOutParameter(3, Types.DECIMAL);         // OUT param:
discounted price

            cs.execute();

            double discountedPrice = cs.getDouble(3);
            System.out.println("Discounted Price: " + discountedPrice);

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- CallableStatement supports both IN and OUT parameters, unlike PreparedStatement.
- IN parameters are set before execution (setString, setDate).

• OUT parameter is declared with registerOutParameter() so JDBC knows its type.

• After execute(), OUT values are read with getXXX().

• This approach is efficient because business rules remain centralized in the DB, reducing Java-side complexity and ensuring consistent calculations.

Future scope:

• Handle multiple OUT parameters if the stored procedure returns more values (like discount breakdowns).

• Use INOUT parameters when the procedure modifies and returns an argument.

• Integrate into a pricing engine service, caching frequent discount results for performance.

Scenario 58: Partial rollback with JDBC savepoints

In an order workflow, you may insert the order header first and then many line items. If a later step fails, you might want to keep the header but discard the problematic lines—without nuking the entire transaction.

Answer:

I'll turn off auto-commit, perform the first set of statements, create a Savepoint, and run downstream operations. If anything breaks after the savepoint, I'll roll back to it (not the start), then commit whatever remains valid.

Code:

```
import java.sql.*;

public class SavepointExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement insOrder = conn.prepareStatement(
                   "INSERT INTO orders (order_id, customer_id) VALUES (?, ?)");
             PreparedStatement insDetail = conn.prepareStatement(
                   "INSERT INTO order_details (order_id, product_id, quantity)
VALUES (?, ?, ?)")) {

            conn.setAutoCommit(false);

            // Step 1: Insert order header
            insOrder.setInt(1, 101);
            insOrder.setInt(2, 5);
            insOrder.executeUpdate();

            // Create a savepoint after header insert
            Savepoint spBeforeDetails = conn.setSavepoint("BeforeOrderDetails");

            try {
                // Step 2: Insert details (could be multiple rows)
                insDetail.setInt(1, 101);
                insDetail.setInt(2, 2);
                insDetail.setInt(3, 5);
                insDetail.executeUpdate();

                // Simulate later failure
                boolean simulateError = true;
                if (simulateError) {
```

```
                throw new SQLException("Simulated failure in later step");
            }

            conn.commit();
            System.out.println("Transaction committed successfully.");
        } catch (SQLException stepErr) {
            // Roll back only the portion after the savepoint
            conn.rollback(spBeforeDetails);
            // At this point: order header kept, details after savepoint undone
            conn.commit(); // commit the valid part (header)
            System.out.println("Rolled back to savepoint, kept order header.");
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }
  }
}
```

Reasoning to interviewer:

- Savepoints give fine-grained control—perfect for multi-step workflows where partial success is acceptable.
- Disabling auto-commit lets me group operations and choose exactly what to keep/rollback.
- Rolling back to the savepoint preserves prior successful steps while discarding the failing segment.
- Using PreparedStatements keeps it safe and fast, and the try-with-resources blocks ensure cleanup.

Future scope:

- Use multiple named savepoints for complex flows (pricing, stock reservation, invoicing).
- Log compensating actions (e.g., release reserved stock) when rolling back.
- Wrap into a service method that returns a result object describing exactly what was persisted and what was rolled back.

Scenario 59: Efficient connection management with a JDBC pool

A busy blogging platform can’t afford to open a fresh DB connection per request. A pool keeps a handful of ready connections, cutting latency and avoiding resource thrash.

Answer:

I’ll configure a DataSource with HikariCP, then obtain connections via ds.getConnection() inside try-with-resources so they’re returned to the pool automatically. Pool sizing and timeouts are tuned for the traffic profile.

Code:

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.*;

public class ConnectionPoolingExample {
    private static HikariDataSource ds;

    static {
        HikariConfig cfg = new HikariConfig();
        cfg.setJdbcUrl("jdbc:mysql://localhost:3306/mydb");
        cfg.setUsername("user");
        cfg.setPassword("password");
        cfg.setMaximumPoolSize(10);
        cfg.setMinimumIdle(2);
        cfg.setConnectionTimeout(30_000);
        cfg.setIdleTimeout(300_000);
        cfg.setPoolName("blog-pool");

        // Optional MySQL prepared-statement cache hints
        // cfg.addDataSourceProperty("cachePrepStmts", "true");
        // cfg.addDataSourceProperty("prepStmtCacheSize", "250");
        // cfg.addDataSourceProperty("prepStmtCacheSqlLimit", "2048");

        ds = new HikariDataSource(cfg);
    }

    public static void main(String[] args) {
        String sql = "SELECT title FROM posts ORDER BY created_at DESC LIMIT
20";
        try (Connection conn = ds.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                System.out.println("Post: " + rs.getString("title"));
            }
```

```
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            if (ds != null) ds.close(); // close pool on app shutdown
        }
    }
}
```

Reasoning to interviewer:

- A DataSource abstracts pooling; borrowing a connection is millisecond-level vs. the heavy TCP/auth cost of new connections.
- HikariCP is lightweight and battle-tested; sizing and timeouts prevent resource exhaustion under load.
- try-with-resources returns connections to the pool reliably, even on exceptions.
- Prepared-statement caching reduces parse/plan overhead for hot queries.

Future scope:

- Externalize pool config per environment (dev/stage/prod).
- Add metrics (Micrometer/Prometheus) to monitor pool utilization and wait times.
- Separate read and write pools (with replicas) for better scalability and isolation.

Scenario 60: Store and retrieve high-resolution images (BLOB) with JDBC

Photo apps often persist originals for backup or moderation. We need a safe way to write large binaries and stream them back out without loading whole files into memory.

Answer:

I'll insert using a PreparedStatement and setBinaryStream(...) (or setBlob(...)) so the driver streams bytes to the DB. For reads, I'll use getBinaryStream() and stream to a file/output stream. try-with-resources guarantees handles are closed even on errors.

Code:

```
import java.io.*;
import java.nio.file.*;
import java.sql.*;

public class BlobExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        Path input = Paths.get("image.jpg");
        Path output = Paths.get("retrieved.jpg");

        insertPhoto(1, input);
        readPhoto(1, output);
    }

    static void insertPhoto(int id, Path file) {
        String sql = "INSERT INTO photos (id, image) VALUES (?, ?)";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql);
             InputStream in = Files.newInputStream(file)) {

            ps.setInt(1, id);
            // If you know the file size, pass it; helps some drivers
            ps.setBinaryStream(2, in, Files.size(file));
            ps.executeUpdate();
            System.out.println("Photo stored for id=" + id);

        } catch (IOException | SQLException e) {
            e.printStackTrace();
        }
    }
```

```
    static void readPhoto(int id, Path outFile) {
        String sql = "SELECT image FROM photos WHERE id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, id);

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    try (InputStream is = rs.getBinaryStream("image")) {
                        Files.copy(is, outFile, StandardCopyOption.REPLACE_EXISTING);
                        System.out.println("Photo retrieved to " + outFile.toAbsolutePath());
                    }
                } else {
                    System.out.println("No photo found for id=" + id);
                }
            }

        } catch (IOException | SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- PreparedStatement streams bytes; we avoid reading the whole file into a byte[].
- getBinaryStream() lets us stream out efficiently; Files.copy handles buffering.
- try-with-resources closes JDBC + I/O resources deterministically, preventing file/LOB handle leaks.
- Passing the known length can improve performance with some drivers.

Future scope:

- Store MIME type and image dimensions alongside the BLOB for faster metadata queries.
- For CDN delivery, consider storing originals in object storage (S3/GCS) and keep only pointers in DB.
- Add checksum (SHA-256) to detect duplicates and verify integrity.

Scenario 61: Stream millions of rows without blowing memory

Analytics jobs shouldn't slurp entire result sets. We want forward-only, server-side cursors and a sensible fetch size so rows are processed in chunks.

Answer:

I'll create a forward-only, read-only PreparedStatement, turn on MySQL cursor fetching (useCursorFetch=true), and set a fetch size (e.g., 1k). Then I'll iterate row-by-row, doing work per row without materializing the whole result.

Code:

```
import java.sql.*;

public class StreamingResultSetExample {
    // Enable MySQL server-side cursors; Connector/J will fetch in pages
    private static final String DB_URL =
"jdbc:mysql://localhost:3306/mydb?useCursorFetch=true";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        String sql = "SELECT id, amount, created_at FROM transactions ORDER BY
id";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS)) {
            conn.setReadOnly(true); // hint for driver/DB

            try (PreparedStatement ps = conn.prepareStatement(
                    sql,
                    ResultSet.TYPE_FORWARD_ONLY,
                    ResultSet.CONCUR_READ_ONLY)) {

                ps.setFetchSize(1000); // page size; tune per workload

                try (ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        int id = rs.getInt("id");
                        double amt = rs.getDouble("amount");
                        Timestamp ts = rs.getTimestamp("created_at");
                        // process row...
                        if (id % 1_000_000 == 0) {
                            System.out.println("Processed up to id " + id);
                        }
                    }
                }
            }
        } catch (SQLException e) {
```

```
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• TYPE_FORWARD_ONLY + CONCUR_READ_ONLY is optimal for streaming: no random access, no updates.

• useCursorFetch=true (MySQL) and setFetchSize(…) make the driver fetch in pages, not all at once.

• setReadOnly(true) signals no writes; some drivers optimize execution plans and memory.

• This pattern protects heap usage and keeps GC pressure low during long scans.

Future scope:

• Use projection (select only needed columns) and proper indexing to reduce I/O.

• Process in parallel by key-range partitioning (e.g., by id or by time windows).

• Add back-pressure/metrics (rows/sec, lag, heap) and checkpointing for resumable jobs.

Scenario 62: Close JDBC resources reliably (no leaks, even on exceptions)

In real apps (APIs, batch jobs), leaked connections or statements slowly starve the pool. I want simple, exception-safe cleanup every time.

Answer:

I use try-with-resources for Connection, PreparedStatement, and ResultSet. For write flows, I also wrap commit/rollback carefully: start a transaction, do the work, commit on success, and in the catch block attempt a rollback in its own try. I'll add a statement timeout to avoid threads hanging forever.

Code:

```
import java.sql.*;
import java.time.LocalDate;
import java.util.logging.Logger;

public class ResourceManagementExample {
    private static final Logger log =
Logger.getLogger(ResourceManagementExample.class.getName());
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mydb";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    // Read-only example (auto-closed in correct order: rs -> ps -> conn)
    public static void listCustomers() {
        String sql = "SELECT name FROM customers ORDER BY name";
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setQueryTimeout(30); // seconds
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    System.out.println(rs.getString("name"));
                }
            }
        } catch (SQLException e) {
            log.severe("Error listing customers: " + e.getMessage());
        }
    }

    // Write example with explicit transaction, commit/rollback handled safely
    public static void addCustomer(String name, LocalDate since) {
        String sql = "INSERT INTO customers(name, since_date) VALUES(?, ?)";
        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS)) {
            conn.setAutoCommit(false);
            try (PreparedStatement ps = conn.prepareStatement(sql)) {
```

```
                ps.setQueryTimeout(30);
                ps.setString(1, name.trim());
                ps.setDate(2, Date.valueOf(since));
                ps.executeUpdate();
                conn.commit();
            } catch (SQLException ex) {
                try { conn.rollback(); } catch (SQLException ignore) {}
                throw ex;
            }
        } catch (SQLException e) {
            log.severe("Insert failed: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        listCustomers();
        addCustomer("Asha", LocalDate.now());
    }
}
```

Reasoning to interviewer:

- try-with-resources guarantees close order (ResultSet → Statement → Connection), even if an exception is thrown mid-loop.
- For writes, rollback lives in its own try so cleanup never throws away the original error.
- Timeouts prevent zombie queries.
- Avoids finally blocks and manual null checks—clean, idiomatic, safe.

Future scope:

- Centralize JDBC helper methods (timeouts, error mapping) or use a small DAO layer.
- Add connection pool (HikariCP) and mark reads setReadOnly(true) for query optimizations.
- Structured logging/metrics around query latency and error rates.

Scenario 63: Fast bulk inserts with PreparedStatement batching

In a bulk catalog import, single-row inserts are slow due to round trips and repeated parsing. Batching fixes that.

Answer:

I'll disable auto-commit, reuse one PreparedStatement, add rows with addBatch(), flush every N rows (e.g., 1,000), and commit per chunk. On any failure, rollback the transaction so we don't get a half-loaded catalog.

Code:

```
import java.sql.*;
import java.util.List;

public class BatchInsertExample {
    private static final String DB_URL =
"jdbc:mysql://localhost:3306/mydb?rewriteBatchedStatements=true";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";
    private static final int BATCH_SIZE = 1000;

    record Product(String name, double price) {}

    public static void main(String[] args) {
        List<Product> items = java.util.stream.IntStream.rangeClosed(1, 10_000)
                .mapToObj(i -> new Product("Product " + i, i * 10.5))
                .toList();
        insertProducts(items);
    }

    public static void insertProducts(List<Product> products) {
        String sql = "INSERT INTO products (name, price) VALUES (?, ?)";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            conn.setAutoCommit(false);
            int count = 0;

            for (Product p : products) {
                ps.setString(1, p.name());
                ps.setDouble(2, p.price());
                ps.addBatch();

                if (++count % BATCH_SIZE == 0) {
                    ps.executeBatch();
                    conn.commit();      // commit this chunk
```

```
                // ps.clearBatch();   // optional; driver usually clears on executeBatch
            }
        }

        if (count % BATCH_SIZE != 0) {
            ps.executeBatch();       // flush remainder
            conn.commit();
        }

        System.out.println("Batch insert completed for " + count + " rows.");
    } catch (SQLException e) {
        e.printStackTrace();         // real apps: log + alert; optionally retry
    }
  }
}
```

Reasoning to interviewer:

- One prepared SQL + many parameter sets drastically reduces network chatter and parsing.
- rewriteBatchedStatements=true (MySQL) lets the driver send multi-values in one go for big wins.
- Chunked commits balance speed with failure isolation.
- Transaction offloads all-or-nothing semantics to the DB.

Future scope:

- Validate data before addBatch; collect rejects with reasons.
- If you need upserts, use INSERT ... ON DUPLICATE KEY UPDATE in the same batched pattern.
- For the absolute fastest path, stage into a temp table then INSERT … SELECT/MERGE.

Scenario 64: Prevent SQL Injection in JDBC

In a signup/login system, string concatenation in queries ("SELECT ... WHERE username='" + input + "'") can allow malicious inputs like "' OR '1'='1" to bypass authentication.

Answer:

Use PreparedStatement with placeholders. The driver treats user input strictly as values, not part of SQL, which prevents injection.

Code:

```
import java.sql.*;

public class SQLInjectionPreventionExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydb";
        String dbUser = "user";
        String dbPass = "password";

        String inputUsername = "admin";
        String inputPassword = "mypassword";

        String sql = "SELECT * FROM users WHERE username = ? AND password =
?";

        try (Connection conn = DriverManager.getConnection(url, dbUser, dbPass);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, inputUsername);
            ps.setString(2, inputPassword);

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    System.out.println("Login successful");
                } else {
                    System.out.println("Invalid credentials");
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- Placeholders ? ensure inputs are bound as literals, not parsed as SQL.
- The driver automatically escapes special characters (', --, ;).
- This avoids vulnerabilities like admin' OR '1'='1.
- Still, passwords should be hashed (e.g., bcrypt) instead of stored in plain text.

Future scope:

- Use strong password hashing (BCrypt/Argon2) and never store raw passwords.
- Apply least-privilege DB users for login modules.
- Add input validation at the API layer for extra safety.

Scenario 65: Work with data offline using RowSet

Sometimes you don’t want a live DB connection (e.g., a desktop app editing inventory offline). A CachedRowSet loads data into memory, lets you manipulate rows, then reconnects later to push updates.

Answer:

I’ll use RowSetProvider.newFactory().createCachedRowSet(), configure DB credentials, execute a query, modify the data in-memory, and finally call acceptChanges() to sync back.

Code:

```
import javax.sql.rowset.CachedRowSet;
import javax.sql.rowset.RowSetProvider;
import java.sql.*;

public class OfflineProductUpdateExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydb";
        String dbUser = "user";
        String dbPass = "password";

        try {
            CachedRowSet rowSet =
RowSetProvider.newFactory().createCachedRowSet();
            rowSet.setUrl(url);
            rowSet.setUsername(dbUser);
            rowSet.setPassword(dbPass);
            rowSet.setCommand("SELECT product_id, product_name, stock_quantity
FROM products");
            rowSet.execute(); // fetch + disconnect

            // offline modifications
            while (rowSet.next()) {
                int currentStock = rowSet.getInt("stock_quantity");
                rowSet.updateInt("stock_quantity", currentStock + 10);
                rowSet.updateRow();
            }

            // push changes back
            rowSet.acceptChanges();
            System.out.println("Offline updates applied successfully.");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- CachedRowSet caches rows locally; connection is closed after fetch.
- We can scroll, update, and even filter in-memory like a disconnected ResultSet.
- Later, acceptChanges() reconnects internally and applies deltas.
- This model reduces DB load and allows users to work offline.

Future scope:

- Use filters/sorting on the RowSet (supports listeners, paging).
- Handle conflict resolution if the same row was updated elsewhere.
- Wrap this logic inside a service so the app can queue offline changes and retry sync later.

Scenario 66: Use Savepoints for Partial Rollback in JDBC

In payment or inventory workflows, we often need checkpoints. For example: deduct from account, then update order. If the second step fails, we shouldn't undo everything—just roll back to the safe point.

Answer:

I'll disable auto-commit, perform the first update, set a savepoint, and then continue with later steps. If a failure occurs, rollback to that savepoint instead of undoing the entire transaction.

Code:

```
import java.sql.*;

public class SavepointExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydb";
        String username = "user";
        String password = "password";

        try (Connection connection = DriverManager.getConnection(url, username,
password)) {
            connection.setAutoCommit(false);

            Statement stmt = connection.createStatement();

            // Step 1: Deduct payment
            stmt.executeUpdate("UPDATE accounts SET balance = balance - 500
WHERE user_id = 101");
            Savepoint sp = connection.setSavepoint("AfterDeduction");

            try {
                // Step 2: Update order status
                stmt.executeUpdate("UPDATE orders SET status = 'PAID' WHERE
order_id = 2001");

                // Commit if everything succeeds
                connection.commit();
                System.out.println("Transaction committed successfully.");
            } catch (SQLException ex) {
                // Roll back only to deduction stage
                connection.rollback(sp);
                connection.commit(); // finalize rollback
                System.out.println("Rolled back to savepoint. Deduction undone, earlier
ops intact.");
            }
        } catch (SQLException e) {
            e.printStackTrace();
```

```
        }
    }
}
```

Reasoning to interviewer:

- setAutoCommit(false) gives manual control.
- A savepoint marks a “checkpoint” after deducting from the account.
- If updating order status fails, rollback(sp) reverts only after that point.
- We finalize with commit() to persist everything before the savepoint.
- This is critical in finance and logistics workflows where partial rollback is better than all-or-nothing.

Future scope:

- Name savepoints by stage (AfterDeduction, AfterTax) for clarity.
- Wrap in try-with-resources for statements.
- In real payments, also add audit logging so failures are traceable.

Scenario 67: Efficient Batch Inserts for Shipments

Daily inserts in thousands can choke performance if done row by row. Instead, batch them.

Answer:

I'll use PreparedStatement.addBatch() and commit in chunks (say every 1,000 records). This reduces round trips and leverages driver optimizations.

Code:

```
import java.sql.*;

public class BatchInsertExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/logistics_db";
        String username = "user";
        String password = "password";

        try (Connection conn = DriverManager.getConnection(url, username,
password)) {
            conn.setAutoCommit(false);

            String sql = "INSERT INTO shipments (shipment_id, destination, weight)
VALUES (?, ?, ?)";
            try (PreparedStatement ps = conn.prepareStatement(sql)) {
                for (int i = 1; i <= 5000; i++) {
                    ps.setInt(1, i);
                    ps.setString(2, "City_" + i);
                    ps.setDouble(3, Math.random() * 100);
                    ps.addBatch();

                    if (i % 1000 == 0) {
                        ps.executeBatch();
                        conn.commit();
                    }
                }

                ps.executeBatch(); // remaining
                conn.commit();
                System.out.println("All shipment records inserted successfully.");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• addBatch() collects statements in memory, executeBatch() sends them in one go.

• Disabling auto-commit avoids per-row commits, improving performance.

• Commit in chunks (1k rows) balances speed with failure safety—if a chunk fails, rollback only that batch.

• Significantly fewer DB round trips compared to 5000 separate inserts.

Future scope:

• Use rewriteBatchedStatements=true (for MySQL) for even faster multi-row inserts.

• Log errors row-wise if some fail (advanced batch error handling).

• For huge volumes, consider bulk loaders (e.g., MySQL LOAD DATA INFILE).

Scenario 68: Retrieving database metadata for auditing

In a banking system, it's common to log database environment details for compliance and troubleshooting. Using DatabaseMetaData, we can query version info, driver details, and even list all tables in the schema.

Answer:

I'll use the DatabaseMetaData interface to fetch database product information and available tables dynamically.

Code:

```
import java.sql.*;

public class DatabaseMetadataExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/bank_db";
        String username = "user";
        String password = "password";

        try (Connection conn = DriverManager.getConnection(url, username,
password)) {
            DatabaseMetaData metaData = conn.getMetaData();

            // Basic product and driver details
            System.out.println("Database Product: " +
metaData.getDatabaseProductName());
            System.out.println("Database Version: " +
metaData.getDatabaseProductVersion());
            System.out.println("Driver Name: " + metaData.getDriverName());
            System.out.println("Driver Version: " + metaData.getDriverVersion());

            // Fetch all table names
            try (ResultSet tables = metaData.getTables(null, null, "%", new
String[]{"TABLE"})) {
                System.out.println("Tables in the database:");
                while (tables.next()) {
                    System.out.println(" - " + tables.getString("TABLE_NAME"));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

• DatabaseMetaData is the go-to interface when an application needs introspection of the DB environment.
• getDatabaseProductName, getDatabaseProductVersion, and getDriverName provide vital auditing info.
• The getTables() method allows us to iterate over schema tables dynamically without hardcoding names.
• This approach is helpful for systems that must run across multiple databases (e.g., MySQL, Oracle) without code changes.

Future scope:

• Extend this to log indexes, primary keys, and foreign keys for schema documentation.
• Add a scheduled job that periodically stores DB metadata snapshots for auditing compliance.
• Use this metadata to build dynamic schema migration or validation tools.

Scenario 69: Dynamically read any query's columns and values

Sometimes you don't know the exact shape of the result (think ad-hoc analytics or admin consoles). You still need to print each row cleanly without hardcoding column names.

Answer:

I'll use ResultSetMetaData to discover column count and labels at runtime, then loop 1..N for each row and call getObject(i) for values. I'll also prefer getColumnLabel so aliases in the SQL show up nicely.

Code:

```
import java.sql.*;
import java.util.StringJoiner;

public class ResultSetMetadataExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/store_db";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        String sql = "SELECT * FROM products"; // could be any query

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {

            ResultSetMetaData meta = rs.getMetaData();
            int cols = meta.getColumnCount();

            // optional: print header
            StringJoiner header = new StringJoiner(" | ");
            for (int i = 1; i <= cols; i++) header.add(meta.getColumnLabel(i));
            System.out.println(header);

            while (rs.next()) {
                StringJoiner row = new StringJoiner(" | ");
                for (int i = 1; i <= cols; i++) {
                    Object value = rs.getObject(i); // handles any SQL type
                    row.add(meta.getColumnLabel(i) + ": " + String.valueOf(value));
                }
                System.out.println(row);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
```

```
}
```

Reasoning to interviewer:

- ResultSetMetaData gives me getColumnCount() and getColumnLabel(i) so the code adapts to any SELECT (with or without aliases).
- getObject(i) returns the most natural Java type per column, so I don't need per-type branching up front.
- This pattern is perfect for generic viewers, debugging dumps, or exporting arbitrary query results.

Future scope:

- Add type-aware formatting (dates, decimals) using getColumnType(i)/getColumnTypeName(i).
- Stream to CSV/JSON; truncate large values with a "more" hint.
- Paginate big results and add a max row cap to protect memory.

Scenario 70: Call a stored procedure with IN/OUT parameters (getStudentGPA)

The DB exposes getStudentGPA(student_id IN, gpa OUT). You need to call it and display the value.

Answer:

I'll use CallableStatement, bind the student ID as an IN param, register the OUT param (use DECIMAL → BigDecimal for precision), execute, then read the OUT value.

Code:

```
import java.math.BigDecimal;
import java.sql.*;

public class CallableStatementExample {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/university_db";
    private static final String DB_USER = "user";
    private static final String DB_PASS = "password";

    public static void main(String[] args) {
        fetchGpa(101); // example student
    }

    static void fetchGpa(int studentId) {
        String call = "{ call getStudentGPA(?, ?) }";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASS);
             CallableStatement cs = conn.prepareCall(call)) {

            cs.setInt(1, studentId);                // IN: student ID
            cs.registerOutParameter(2, Types.DECIMAL); // OUT: GPA
(DECIMAL/NUMERIC)

            cs.execute();

            BigDecimal gpa = cs.getBigDecimal(2);
            if (cs.wasNull()) {
                System.out.println("No GPA available for student " + studentId);
            } else {
                // If you prefer double: gpa.doubleValue()
                System.out.println("GPA for student " + studentId + ": " + gpa);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
```

```
}
```

Reasoning to interviewer:

- CallableStatement is the right API for procedures; IN params via setXxx, OUT params via registerOutParameter.
- Reading DECIMAL as BigDecimal avoids precision loss that double could introduce.
- cs.wasNull() lets me distinguish between zero and a NULL GPA.
- This cleanly reuses DB-side logic and keeps Java service code thin.

Future scope:

- Return additional OUT params like credit count or academic standing.
- Wrap calls in a DAO/service; map errors to domain exceptions.
- Add connection pooling + timeouts for resilience in production.

Scenario 71: Secure product search against SQL injection

When users type search terms (like *Nike Shoes*), you want flexibility (partial matches with LIKE) but also safety against malicious input like "' OR 1=1 --".

Answer:

I'll use a PreparedStatement with placeholders (?) so the JDBC driver escapes input. Then I'll wrap the search term in % for flexible matching.

Code:

```
import java.sql.*;
import java.util.Scanner;

public class SQLInjectionSafeSearch {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/retail_db";
        String username = "user";
        String password = "password";

        try (Connection conn = DriverManager.getConnection(url, username,
password);
             Scanner sc = new Scanner(System.in)) {

            System.out.print("Enter product name to search: ");
            String productName = sc.nextLine();

            String sql = "SELECT product_id, product_name, price FROM products
WHERE product_name LIKE ?";
            try (PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setString(1, "%" + productName + "%"); // safe parameter binding

                try (ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        int id = rs.getInt("product_id");
                        String name = rs.getString("product_name");
                        double price = rs.getDouble("price");
                        System.out.println(id + " - " + name + " ($" + price + ")");
                    }
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Reasoning to interviewer:

- Using PreparedStatement instead of string concatenation ensures user input is treated as a value, not SQL code.
- Wrapping the term in % gives flexible “contains” searches while still safe.
- This protects against classic attacks like ' OR 1=1.
- Printing id, name, price keeps results meaningful for a retail context.

Future scope:

- Add pagination (LIMIT ? OFFSET ?) to handle large result sets.
- Use case-insensitive search (LOWER(product_name) LIKE ?).
- Combine with full-text indexes for better performance on big catalogs.